

ResearchDraft – AI Powered Research Paper Writing Tool

M Hamza Khalid Kiani (BCS203225)

Muhammad Haroon (BCS203230)

Kh. Hasan Nisar (BCS203234)



Spring-2024

Supervised By:

Dr. Shafiq Ur Rehman Khan

Department of Computer Science

Capital University of Science & Technology, Islamabad

PROJECT REPORT



Version	V 3.0	NUMBER OF MEMBERS	3
TITLE	Research Draft: AI-Powered Research Paper Writing Tool		
SUPERVISOR NAME	Dr. Shafiq Ur Rehman Khan		

MEMBER NAME	REG. NO.	EMAIL ADDRESS
M Hamza Khalid Kiani	BCS203225	BCS203225@cust.pk
Muhammad Haroon	BCS203230	BCS203230@cust.pk
Kh. Hasan Nisar	BCS203234	BCS203234@cust.pk

MEMBERS' SIGNATURES	
_____	Supervisor's Signature

APPROVAL CERTIFICATE

This project, entitled as “ResearchDraft – AI Powered Research Paper Writing Tool” has been approved for the award of

Bachelor of Science in Computer Science

Committee Signatures:

Supervisor:

(Dr. Shafiq Ur Rehman)

Project Coordinator:

(Mr. Bilal Ahmed)

Head of Department:

(Dr. Abdul Basit)

DECLARATION

We, hereby, declare, “No portion of the work referred to, in this project has been submitted in support of an application for another degree or qualification of this or other institution of learning”. It is further declared that this undergraduate project, neither as a whole nor as a part there of has been copied out from any sources, wherever references have been provided.

MEMBERS' SIGNATURES

ACKNOWLEDGEMENTS

We are grateful of our parents who are investing so much on us that we are here today. We are obliged that Dr. Shafiq Ur Rehman Khan has undertaken us in his supervision and invested his time and efforts in overseeing our progress and guiding us to put the best. We really appreciate that he provided the best guidance to meet the objective of our FYP. Moreover, we are highly thankful to Mr. Bilal Ahmed for addressing our every query and guiding us in the light of his best knowledge, regarding the overall FYP's formal undertakings as well as our concerns regarding FYP during this entire epoch.

EXECUTIVE SUMMARY

This project, "Research Draft: AI-Powered Research Paper Writing Tool" aims to revolutionize the academic writing process for students, researchers, and academicians by integrating advanced AI-driven suggestions with a user-friendly interface. Addressing the common challenges of time-consuming research, tedious revisions, and context-sensitive content optimization, Research Draft offers a seamless solution that enhances productivity and ensures scholarly excellence. The core innovation lies in its ability to scrape user-specified web pages for reference research papers and leverage state-of-the-art AI models, like to generate tailored suggestions that improve the quality and coherence of research paper of the user. This approach not only saves valuable time but also provides personalized content adjustments and user control, ensuring that each paper meets high academic standards. The front-end interface, developed using HTML, CSS, and JavaScript, offers a smooth user experience, while the back-end, built with Python and in Django framework, handles web scraping, AI integration, and data management efficiently. The project's benefits are vast, including streamlined research processes, improved academic output, and significant time savings, making it a valuable tool for the academic community. This comprehensive tool not only empowers users to achieve their academic goals with ease but also fosters a more efficient and productive research environment.

Table of Contents

Contents

Chapter 1	1
1.1. Project Introduction.....	1
1.2. Existing Examples / Solutions.....	3
1.3. Business Scope	4
1.4. Useful Tools, Technologies, and Methods	5
1.5. Project Work Break Down	7
1.6. Project Timeline.....	8
1.7. Summary:	9
Chapter 2	10
Requirement Specification and Analysis.....	10
2.1. Functional Requirements.....	10
2.2. Non – Functional Requirements.....	12
2.3. Selected Functional Requirements	13
2.4. System Use Case Modeling	14
2.5. System Sequence diagrams	23
2.6. Domain Model	30
Chapter 3 System Design.....	31
3.1. Software Architecture	31
3.2. Class Diagram	33
3.3. Sequence Diagram	35
3.4. Entity Relationship Diagram.....	39
3.5. Database Schema	41
3.6. User Interface Design	42

Chapter 4	47
Software Development.....	47
4.1. Coding Standards.....	47
4.2. Development Environment.....	48
4.3. Software Description	48
Chapter 5	54
Software Testing	54
5.1. Testing Methodology	54
5.2. Testing Environment	55
5.3. Testing Cases.....	55
Chapter 6	63
Software Deployment.....	63
6.1. Installation / Deployment Process Description	63
Chapter 7	66
Project Evaluation	66
References	67
Plagiarism Report.....	68

List of Figures

Figure 1.1: Proposed System.....	2
Figure 1.2: Project Work Breakdown.	7
Figure 1.3: Proposed Gantt Chart	8
Figure 2.1: Use Case Diagram	14
Figure 2.2: SSD on Create Account.....	23
Figure 2.3: SSD on Login Account.....	24
Figure 2.4: Create New Project or Access Previous Project	25
Figure 2.5: SSD on Uploading Reference Research Paper links	26
Figure 2.6: SSD on View Extracted Content	27
Figure 2.7: SSD on Sending prompt	28
Figure 2.8: SSD on View Suggestions	28
Figure 2.9: SSD on Edit Text in text editor	29
Figure 2.10: Domain Model.....	30
Figure 3.1: System Architecture Diagram.....	32
Figure 3.2: Class Diagram	34
Figure 3.3: Sequence Diagram - Login	35
Figure 3.4: Sequence Diagram – Uploading Research Paper links.....	36
Figure 3.5: Sequence Diagram – Viewing Extracted Content	37
Figure 3.6: Sequence Diagram – Suggestions.....	38
Figure 3.7: Entity Relationship Diagram	40
Figure 3.8: Database Schema.....	41
Figure 3.9: Website - Home Page.....	42
Figure 3.10: Website - Features.....	43
Figure 3.11: Website - Pricing.....	43
Figure 3.12: Website - Blogs.....	44
Figure 3.13: Website - Sign up.....	44
Figure 3.14: Website - Login.....	45
Figure 3.15: Website - Main dashboard.....	45
Figure 3.16: Website - Upload Paper links.....	46
Figure 3.17: Website - View Content.....	46

List of Tables

Table 1.1: Comparison of Existing AI Tool.....	3
Table 2.1: Functional Requirement.....	10
Table 2.2: Non - Functional Requirement.....	12
Table 2.3: Selected Functional Requirement.....	13
Table 2.4: Use case description of Creating Account.....	15
Table 2.5: Use case description of Login into account.....	16
Table 2.6: Use case description for creating new project.....	17
Table 2.7: Use case description for uploading reference research paper links.....	18
Table 2.8: Use case description of viewing extracted content.....	19
Table 2.9: Use case description of sending prompt to acquire suggestions.....	20
Table 2.10: Use case description of viewing Suggestion.....	21
Table 2.11: Use case description of Edit text in text editor.....	22
Table 5.1: Test Case 1: Create Account.....	55
Table 5.2: Test Case 2 - Login Account.....	56
Table 5.3: Test Case 3 - Create new project.....	57
Table 5.4: Test Case 4 – Upload reference webpage links.....	58
Table 5.5: Test Case 5 – View Extracted Content.....	59
Table 5.6: Test Case 6 – Send prompt.....	60
Table 5.7: Test Case 7 – View Suggestions.....	61
Table 5.8: Test Case 8 – Edit research paper content.....	62

List of Abbreviations

1. **AI:** Artificial Intelligence
2. **API:** Application Programming Interface
3. **CLI:** Command Line Interface
4. **CSS:** Cascading Style Sheets
5. **ERD:** Entity-Relationship Diagram
6. **HTML:** Hypertext Markup Language
7. **IEEE:** Institute of Electrical and Electronics Engineers
8. **LLM:** Large Language Model
9. **NLP:** Natural Language Processing
10. **OTP:** One-Time Password
11. **SSD:** System Sequence Diagram
12. **URL:** Uniform Resource Locator

Blank Page

Chapter 1

1.1. Project Introduction

In the current highly competitive academic environment, there is an increased emphasis on producing well-structured, well-articulated, and unique research papers. Nevertheless, for novice academic writers, crafting a research paper can be a daunting task.

Research papers are formal papers that communicate the results of a study, inquiry, or research effort carried out by an individual or team of researchers. They play a crucial role in academic and scientific exchange and are commonly published in academic journals, presented at conferences, or shared in other scholarly environments. These articles aim to add fresh insights, share research methods, and promote discourse within a particular area of study [1].

This project aims to create an AI-driven tool for academic paper writing, intended to assist students and researchers in improving the quality of their research papers. The tool will utilize advanced techniques such as Large Language Models, Natural Language Processing, and Machine Learning to analyze uploaded reference documents and offer intelligent recommendations for overall improvement.

The aim for this project is to create a smart AI tool that helps users generate and enhance research papers. In the midst of an academic environment flooded with data, this tool will aid users in efficiently managing the process of creating research papers. Key functionalities of the system will include:

- Users will be able to upload reference paper links, which the system will process meticulously. It will break down these research papers into distinct components, extracting titles, headings, text.
- The extracted components will be uploaded and stored on the DJANGO server using file handling techniques facilitating easy retrieval and manipulation of research paper elements.
- Leveraging LLM and NLP techniques, the system will analyze the stored data to provide users with valuable suggestions for optimizing their research papers.

By incorporating these features, this AI-driven tool aims to empower researchers and scholars to produce research papers that are not only well-structured but optimized as well.

In Figure 1.1, the methodology of the project is shown in which all the steps of the AI - powered research paper writing tool are displayed clearly. The steps include:

Reference Paper Links: In this step the user will upload reference research paper links (IEEE Xplore) on the website.

Web Scrapping: This step involves breaking down of the user provided links into smaller, more manageable pieces and text is divided section wise.

Preprocessing: This step involves preparing the data that will be used in the research paper writing. This process encompasses tasks such as removing equations, tables from the data as these are not required.

File Handling: After collecting and preprocessing the data, it is uploaded on the DJANGO server using file handling techniques.

Customizable LLM: This step involves using Customizable LLM on the uploaded reference research papers. This LLM will be used in the process of giving prompt and generating response/suggestions.

Prompts: The user will send prompts to the LLM to enhance the overall quality of their own research paper.

Response: This step involves using the customizable LLM to generate output for improving the research paper.

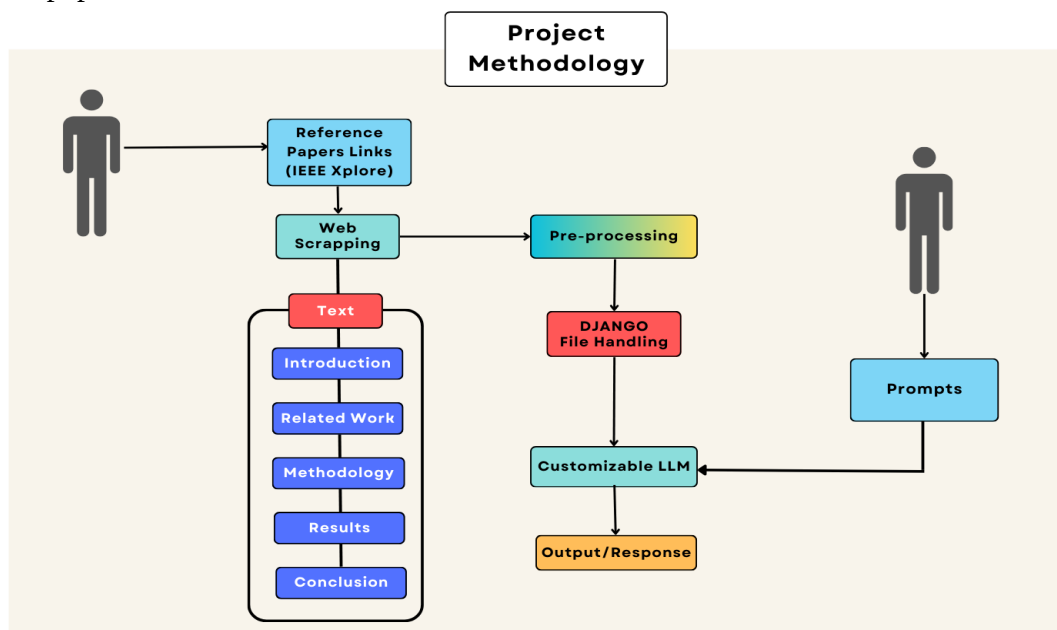


Figure 1.1: Proposed System

As we embark on this final year project, we anticipate that it will contribute significantly to the academic community by simplifying and enhancing the research paper writing process.

1.2. Existing Examples / Solutions

In Table 1.1, we compared several existing AI writing tools on different features. Based on this comparison, we have designed our project idea to address the shortcomings of these tools and enhance the overall academic writing process.

Table 1.1: Comparison of Existing AI Tools

Features	Smodin AI	Jenni AI	Google Bard AI	Textor AI	Jasper AI
Grammar Check	Yes	Yes	No	Yes	Yes
Sentence Structure	Yes	Yes	Yes	No	Yes
Business Model	Free, Premium	Free, Premium	Free	Free, Premium	Paid
Writing Styles	Direct	Creative	Thorough	Conversational	Clear
AI Capabilities	AI Copywriting, Grammar and Style Suggestions	AI Copywriting, Content Spinning, Grammar, Style Suggestions	AI Copywriting, Content Spinning, Grammar and Style Suggestions	AI Copywriting, Content Generation	AI Copywriting, Grammar and Style Suggestions
Integrations	CMS	Automation platforms,	Google products	Customer service platforms	Content writing tools
Customization	High	Limited	Limited	High	High
Learning Curve	Gradual	Moderate	Moderate	Moderate	Moderate
Additional Features	Concise and straight forward writing style	Ability to generate imaginative and original text	Comprehensive and informative writing	Create a sense of connection with the reader.	Well-written and well-structured text
Privacy and Security	Secure	Secure	Secure	Secure	Secure
Mobile App	No	Yes	No	No	No

1.3. Business Scope

An AI-driven tool for composing research papers holds substantial potential. Students, academicians and researchers alike require the ability to produce top-notch academic papers, and an AI tool can facilitate this process with greater speed and effectiveness.

The following are some of the potential target markets for the tool:

Students: Students at every educational level, whether at the undergraduate or postgraduate level, can gain advantages from utilizing an AI-based tool for writing research papers. This resource aids students in honing their research writing abilities and generating research papers of high quality.

Researchers: Researchers must also produce high-quality research papers to publish their findings and obtain funding. The utilization of an AI-driven tool for writing research papers can aid researchers in saving time, decreasing efforts, and enhancing the overall quality of their work.

Academicians: Academicians must also produce high-quality research papers to advance their careers. This tool can save time and effort while enhancing the quality of their work.

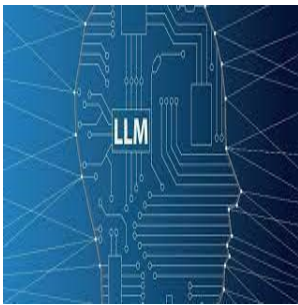
The tool can assist users in enhancing the structure, content, and grammar of their research papers. This could result in higher-quality research papers that are more likely to be accepted by academic journals and conferences.

1.4. Useful Tools, Technologies, and Methods



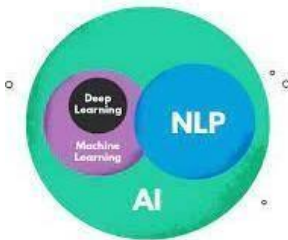
Python

Python is a high-level programming language, which is used for general purpose. It is simple and easy, and holds numerous programming concepts: structured, functional, and object-oriented.



LLM

Large Language Models (LLMs) are known for their natural language understanding and generation capabilities, which can be highly valuable in the context of academic and research paper writing.



NLP

Natural Language Processing techniques are essential for understanding and processing the text, figures, equations, and other components of research papers.

Django



Web frameworks can help streamline the development of the user interface and backend components of your application. A Python framework called Django makes it simpler to develop websites, it follows the "batteries-included" philosophy, meaning it provides a wide range of built-in features and libraries that can accelerate web application development.



Visual Studio Code

For Windows, Linux, and macOS, Microsoft created Visual Studio Code, popularly known as VS Code, a source-code editor. It is compatible with a wide range of programming languages, including Python, JavaScript and other languages commonly used.



Visual Paradigm

Visual Paradigm:

Visual Paradigm features a rich set of Agile and Scrum tools for project management. This a tool used for creating & managing the UML diagrams or the unified modeling language cases.



Microsoft Word

Microsoft Word is a useful tool for creating professional-looking documents. It functions as a basic text editor. For the documentation of this project, MS Word is being used.

1.5. Project Work Break Down

In Figure 1.2, work breakdown of the project is displayed.

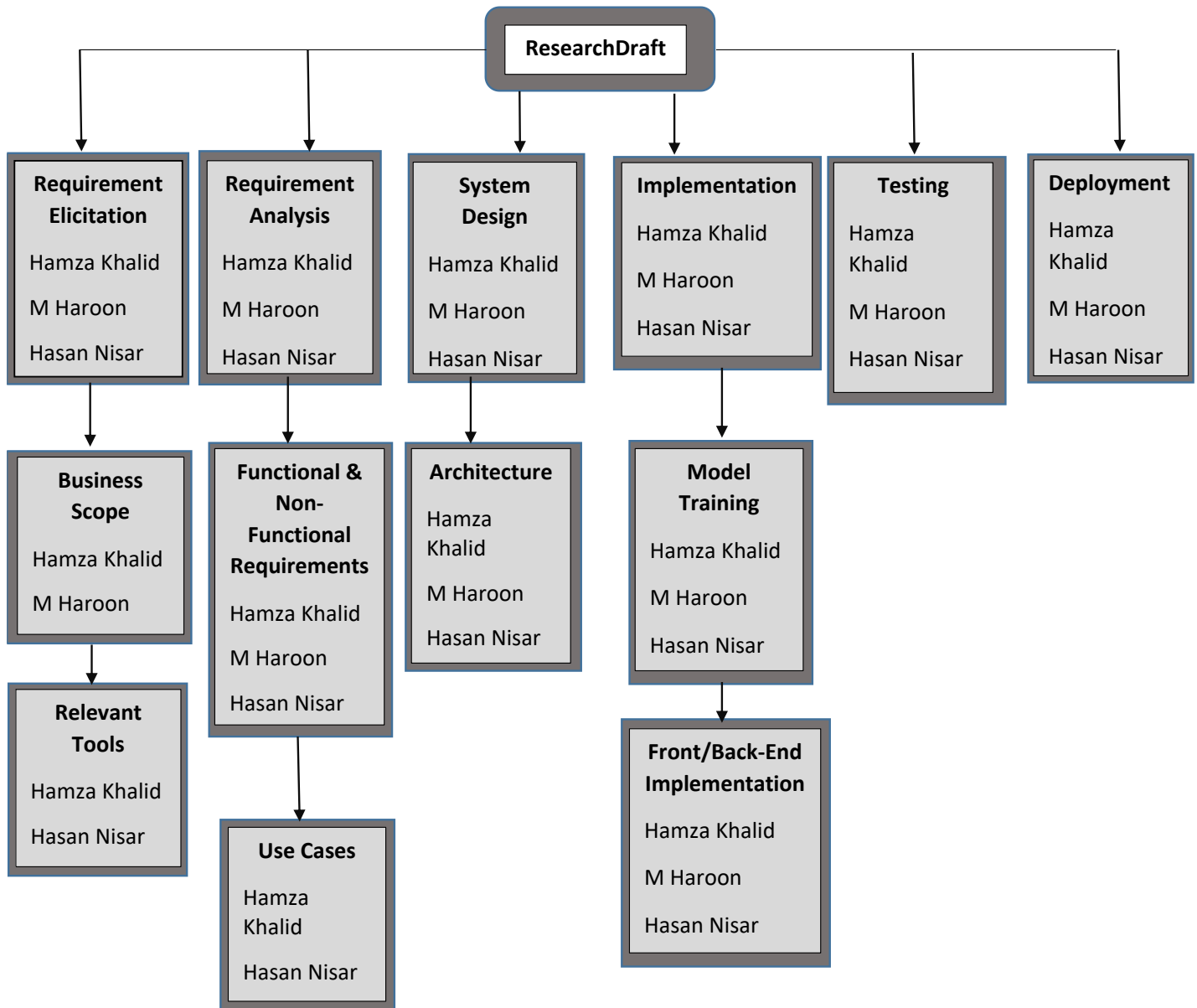


Figure 1.2: Project Work Breakdown

1.6. Project Timeline

Every phase has a specific time so the project can be completed in a specific duration of time. The project is going to start in October 2023 and will be completed in July 2024. In Figure 1.3, a Gantt Chart is designed to show the project duration more specifically

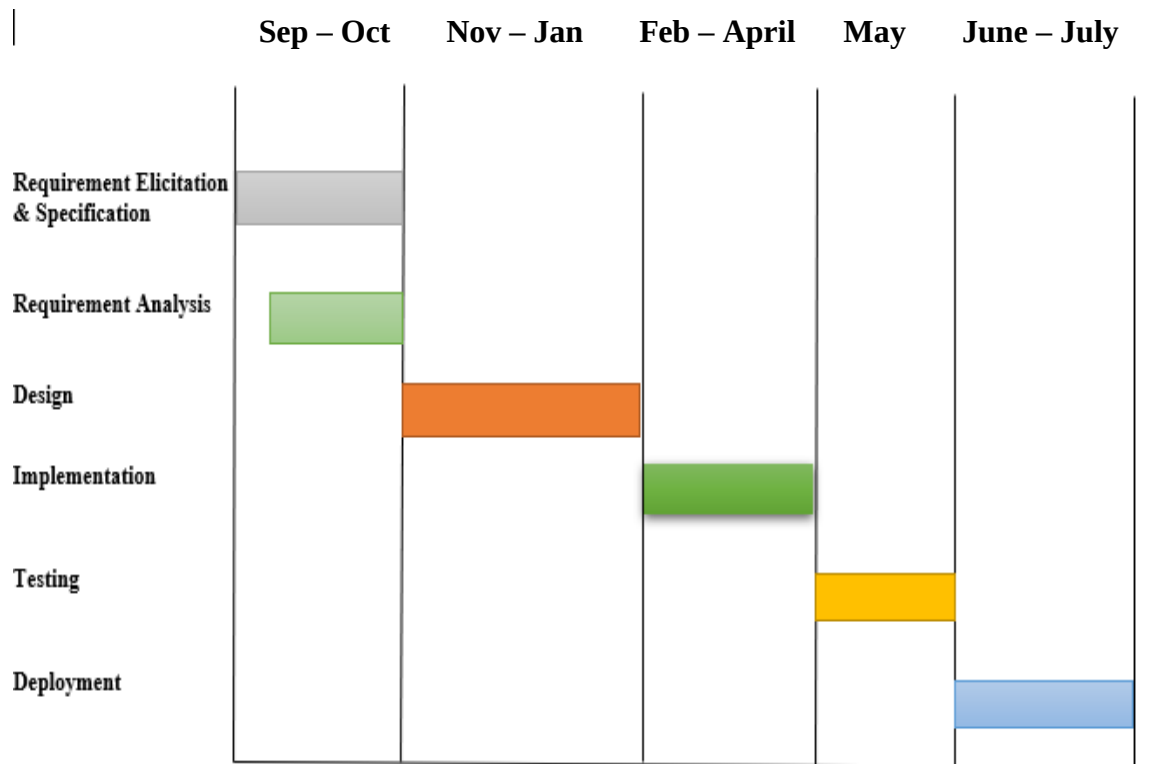


Figure 1.3: Proposed Gantt Chart

1.7. Summary:

The goal of the project is to create an AI-Powered Tool for writing research papers, an advanced system intended to transform the process of creating research papers. Users have the ability to upload reference documents in the form of links, which are then analyzed by the system and organized into separate components. These components are uploaded and stored on the DJANGO server.

Leveraging advanced customizable LLM techniques, the tool offers users intelligent recommendations to enhance their research papers, optimize content structure, and refine language usage.

The project's scope includes a wide range of functionalities and targets researchers, scholars, and academic institutions. Its goal is to simplify the process of writing research papers and improve academic performance.

Chapter 2

Requirement Specification and Analysis

Requirement specification and analysis involves identifying the needs of the user as well as their anticipated application requirements [2]. This requires regular interaction with system users to establish precise feature expectations and address any conflicts or uncertainties in the requirements.

Stakeholder engagement was crucial in shaping the project's requirements defined in Table 2.1, ensuring alignment with the diverse perspectives within the user community. We conducted meetings with professors and researchers to understand the challenges they face in writing research papers. Additionally, we gathered insights on the difficulties encountered during the research paper writing process from both undergraduate and postgraduate students to better comprehend their specific pain points and expectations.

2.1. Functional Requirements

Functional requirements are the core functionality of a system. These requirements collectively define the fundamental features of the AI tool, ensuring that it not only streamlines the process of writing research papers but also delivers a customized and efficient user experience.

The following Table 2.1 outlines the key functional requirements identified through stakeholder engagement, interviews, and analysis of the project's objectives.

Table 2.1: Functional Requirements

S. No.	Functional Requirement	Type	Status
1	User shall be able to create an account with a username, password and email address.	Core	Completed
2	User shall be able to login into the account.	Core	Completed
3	User shall be able to create new project or access previous projects if already created any project.	Core	Completed

4	User shall be able to upload reference research papers in the form of webpage links.	Core	Completed
5	User shall be able to view extracted content from the uploaded research papers section wise.	Core	Completed
6	User shall be able to send prompts for acquiring suggestions.	Core	Completed
7	User shall be able to view the suggestions for improvement generated by the system.	Core	Completed
8	User shall be able to type or paste research paper content in the text editor to edit the content.	Core	Completed
9	User shall be able to export the edited text in TXT file format.	Supporting	Completed

2.2. Non – Functional Requirements

Non-functional requirements enhance overall quality of a system. One important reason for incorporating NFRs is to determine how effectively the system will carry out specific functions [3]. Non-functional requirements contribute to enhancing the efficiency of a system. The non-functional requirements listed in Table 2.2 tackle critical quality concerns for software systems, encompassing aspects such as scalability, maintainability, performance, security, and reliability.

Table 2.2: Non - Functional Requirements

S. No.	Non-Functional Requirements	Category
1	The web interface should be designed with focusing on ease of use, and an aesthetically pleasing design to enhance the user experience.	Usability
2	The system shall be able to provide in a timely manner.	Performance
3	The system shall be able to handle a large number of users and documents.	Scalability
4	The system shall be able to protect user data from unauthorized access.	Security
5	The system shall be available and accessible to users when needed.	Availability
6	The system shall be compatible on multiple web browsers and document formats.	Compatibility
7	The system shall be easy to maintain and update.	Maintainability

2.3. Selected Functional Requirements

Table 2.3: Selected Functional Requirements

S. No.	Functional Requirement	Type
1	User shall be able to create an account with a username, password and email address.	Core
2	User shall be able to log-in into the account.	Core
3	User shall be able to create new project or access previous projects if already created any project.	Core
4	User shall be able to upload reference research papers in the form of webpage links.	Core
5	User shall be able to view extracted content from the uploaded research papers.	Core
6	User shall be able to send prompts for acquiring AI-driven suggestions.	Core
7	User shall be able to view the suggestions for improvement generated by the system.	Core
8	User shall be able to type or paste research paper content in the text editor to edit the content.	Core

2.4. System Use Case Modeling

The system's behavior description is represented through a use case diagram, which outlines the interactions between users and the system by defining their objectives.

The use case diagram of our project in Figure 2.1 provides a visual representation of the interactions between the system and its potential user, showcasing the primary functionalities and user interactions.

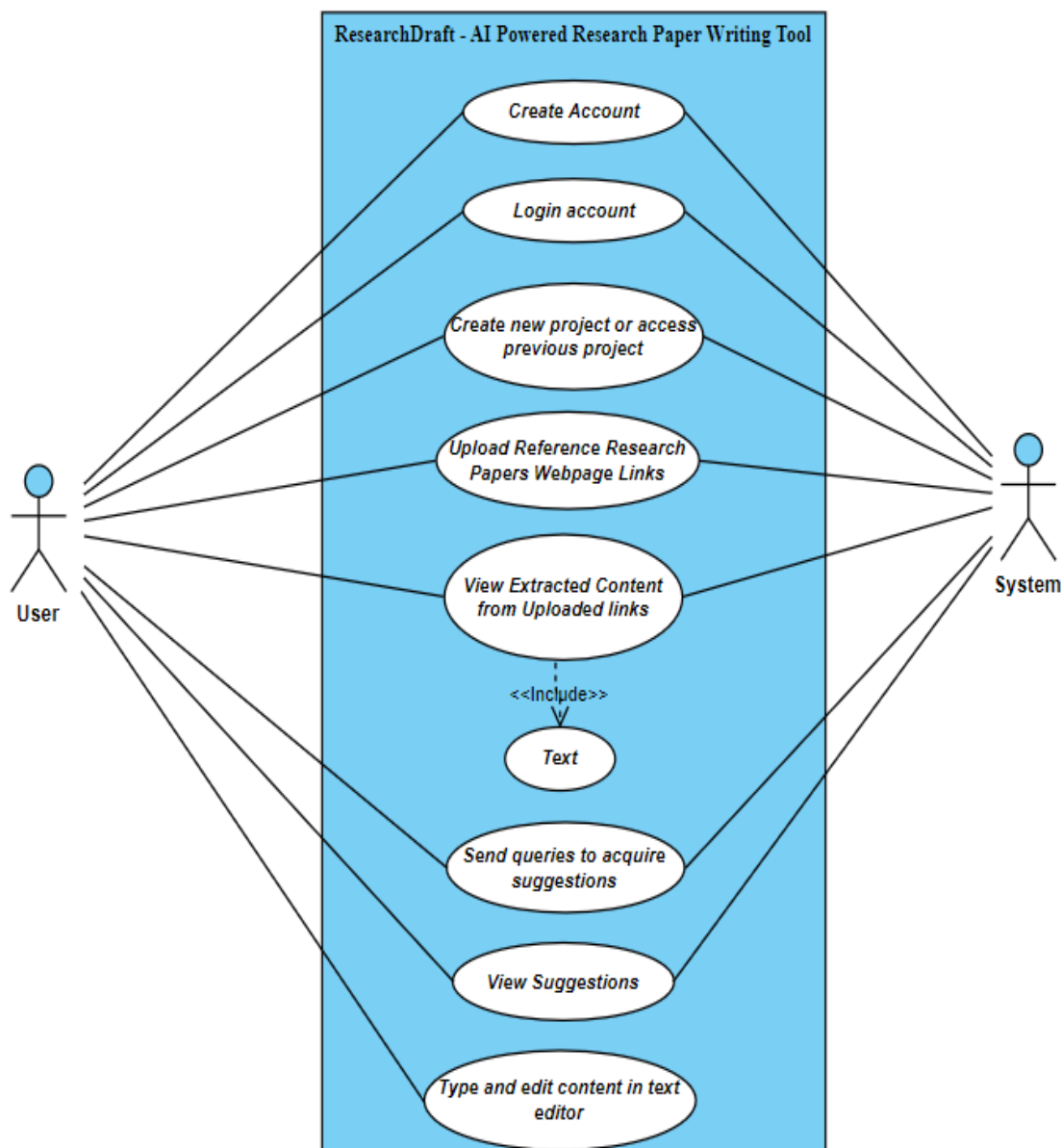


Figure 2.1: Use Case Diagram

Following are use cases against the selected functional requirements from Table 2.3:

Use Case 1 Create Account:

The use case in Table 2.4 outlines the process for the Create Account module, detailing how the system will function when a user creates an account.

Table 2.4: Use case description of Creating Account

Use Case ID:	01		
Use Case Name:	Create an account on the website.		
Created By:	M Haroon	Last Updated By:	M Haroon
Date Created:	01/11/23	Last Revision Date:	15/07/24
Actors:	User		
Description:	Create an account with a username, password and email address		
Trigger:	Create account button		
Preconditions:	User must be on the webpage		
Post conditions:	User account will be created		
Normal Flow:	Actor	System	
	1. User click on Create account button.	2. System will display the form on screen.	
	3. User will add his/her personal details to create account.		
	4. User will then click on submit button		
		5. System will generate OTP and send to user email address	
	6. User will enter OTP within 5 minutes before it expires.	7. Account will be verified and created.	
Alternative Flows:	Account should not be created if any credential is missing or even if OTP is not entered.		
Exceptions:	User did not receive OTP in email.		

Use Case 2 Login into account:

The use case in Table 2.5 outlines the process for the Login module, detailing how the system will function when a user login into account.

Table 2.5: Use case description of Login into account

Use Case ID:	02		
Use Case Name:	Login into account.		
Created By:	M Hamza Khalid	Last Updated By:	M Hamza Khalid
Date Created:	01/11/23	Last Revision Date:	06/05/24
Actors:	User		
Description:	User will Login into the account.		
Trigger:	Press Login button		
Preconditions:	User must have a verified account.		
Post conditions:	User will be logged into their account.		
Normal Flow:	Actor	System	
	1. User will click on log in button.	2. System will display email and password option	
	3. User will provide email address and password.		
	4. User will click on submit button	5. System will open users account.	
	6. User will be logged into their account.		
Alternative Flows:	User will enter wrong username or password.		
Exceptions:	User clicks on any wrong button.		

Use Case 3 Create New Project or Access Previous Projects:

The use case in Table 2.6 outlines the process for the Create New Project module, detailing how the system will function when a user creates a new project or access previous project.

Table 2.6: Use case description for creating new project

Use Case ID:	03		
Use Case Name:	Create new project or access previous projects.		
Created By:	M Haroon	Last Updated By:	M Haroon
Date Created:	15/06/24	Last Revision Date:	20/06/24
Actors:	User		
Description:	User will create new project or access previous projects.		
Trigger:	Click on Create New Project button.		
Preconditions:	User must be logged in into their account		
Post conditions:	User will be able to create new project or access previous projects.		
Normal Flow:	Actor	System	
	1. User click on create new project button	2. A pop-up will display to enter project details.	
	3. User will enter project name, domain and then click on save button		
		4. System will save the information and redirect to next page.	
	5. In case of previous project, user can access previous projects and update it using action buttons.	6. System will redirect to the webpage as selected by user.	
Alternative Flows:	User did not enter all information regarding project details.		
Exceptions:	User did not create any project.		

Use Case 4 Upload Reference Research Papers links:

The use case in Table 2.7 outlines the process for the uploading reference links (IEEE Xplore), detailing how the system will function when a user enters reference research paper links.

Table 2.7: Use case description for uploading reference research paper links

Use Case ID:	04		
Use Case Name:	Upload reference research paper links on the website		
Created By:	M Haroon	Last Updated By:	M Haroon
Date Created:	1/11/23	Last Revision Date:	10/03/24
Actors:	User		
Description:	User will upload reference research paper links on the website.		
Trigger:	Click on Enter Reference Paper link input box.		
Preconditions:	User must be logged in into their account		
Post conditions:	User will be able to upload reference research paper links.		
Normal Flow:	Actor	System	
	1. User click on input box and enter links (IEEE Xplore).		
	2. User will click on save button.		
		3. System will display the Processing sign.	
		4. System will accept the links.	
Alternative Flows:	User choose wrong format for webpage links so the links will not be saved.		
Exceptions:	User did not enter any links.		

Use Case 5 View Extracted Content:

The use case in Table 2.8 outlines the process for the View extracted content module, detailing how the system will function when a user views research papers content section wise and download it.

Table 2.8: Use case description of viewing extracted content from uploaded links

Use Case ID:	05		
Use Case Name:	View extracted content from the uploaded research papers.		
Created By:	M Haroon	Last Updated By:	M Haroon
Date Created:	1/11/23	Last Revision Date:	15/07/24
Actors:	User		
Description:	User shall be able to view extracted content from the uploaded research paper links and download separate sections.		
Trigger:	Press View Extracted Content button.		
Preconditions:	User must have uploaded reference research paper links on the webpage.		
Post conditions:	User will be viewing content from the uploaded research paper links and download separate sections.		
Normal Flow:	Actor	System	
	1. User click on View Content button.	2. System will display the Paper Name and text button on screen	
	3. User will select Text button.	4. System will display drop down menu containing headings of research paper.	
	5. User will select any option from the drop-down menu.		
		6. System will display text within that section.	
	7. User clicks on download icon in the drop-down menu.	8. System will download TXT file having content of that specific section.	
Alternative Flows:	User don't choose view content option.		
Exceptions:	User didn't upload reference webpages links.		

Use Case 6 Send Prompt to acquire suggestions:

The use case in Table 2.9 outlines the process for the sending prompt to acquire suggestions module, detailing how the system will function when a user sends prompt to get response.

Table 2.9: Use case description of sending prompt to acquire suggestions

Use Case ID:	06		
Use Case Name:	Write and send prompt to acquire suggestions		
Created By:	Kh. Hasan Nisar	Last Updated By:	Kh. Hasan Nisar
Date Created:	15/06/24	Last Revision Date:	07/07/24
Actors:	User		
Description:	User will write and send prompt to acquire suggestions.		
Trigger:	Press Send button on the webpage		
Preconditions:	User must have uploaded reference research paper links on the webpage.		
Post conditions:	User will be getting suggestions for their own research paper.		
Normal Flow:	Actor	System	
	1. User write the prompt in the input box.		
	2. User clicks on the send button.		
		3. System will process the prompt entered by user.	
Alternative Flows:	User don't write any prompt.		
Exceptions:	Nil		

Use Case 7 View Suggestions:

The use case in Table 2.10 outlines the process for the view response generated by the system, detailing how the system will function when a user views response or suggestions against the entered prompt.

Table 2.10: Use case description of Viewing Suggestions

Use Case ID:	07		
Use Case Name:	View the suggestions for improvement of research paper generated by the system.		
Created By:	M Haroon	Last Updated By:	M Haroon
Date Created:	01/11/23	Last Revision Date:	15/06/24
Actors:	User		
Description:	User will be able to view the suggestions for improvement generated by the system.		
Trigger:	Press Send button on the webpage		
Preconditions:	User must have entered prompt in the input box		
Post conditions:	User will be able to view the suggestions for improvement of researchpaper.		
Normal Flow:	Actor	System	
	1. User click on send button.		
		2. System will display the response on screen.	
Alternative Flows:	User enters prompt that are not linked with the research paper content.		
Exceptions:	User did not enter any prompt.		

Use Case 8 Edit Text in Text Editor

The use case in Table 2.11 outlines the process for the view response generated by the system, detailing how the system will function when a user views response or suggestions against the entered prompt.

Table 2.11: Use case description of Edit text in text editor

Use Case ID:	08		
Use Case Name:	Type and edit research paper content in the text editor.		
Created By:	M Haroon	Last Updated By:	M Haroon
Date Created:	18/06/24	Last Revision Date:	24/06/24
Actors:	User		
Description:	User will be able to type and edit research paper content in the text editor.		
Trigger:	Press Text Editor button		
Preconditions:	User must have created a project or accessed any previous project.		
Post conditions:	User own research paper content will be updated.		
Normal Flow:	Actor	System	
	1. User click on Text editor button.		
		2. System will display text editor.	
	3. User will type or paste text and update it.		
	4. User will download TXT file once content is updated.		
Alternative Flows:	User will not download TXT file rather just copy the text.		
Exceptions:	Nil		

2.5. System Sequence diagrams

System Sequence Diagrams are a variant of sequence diagrams found in the Unified Modeling Language. They are designed to illustrate the chronological order of interactions between users and the system in order to accomplish a specific action or use case [2]. System sequence diagrams typically correspond with the normal flows outlined in use case descriptions.

Create Account:

The sequence diagram captures the dynamic aspects of the "Create Account" use case, showcasing the interactions between the user and the system. It offers a clear understanding of the steps from Table 2.4 where the normal flow is presented for the user initiating the account creation process.

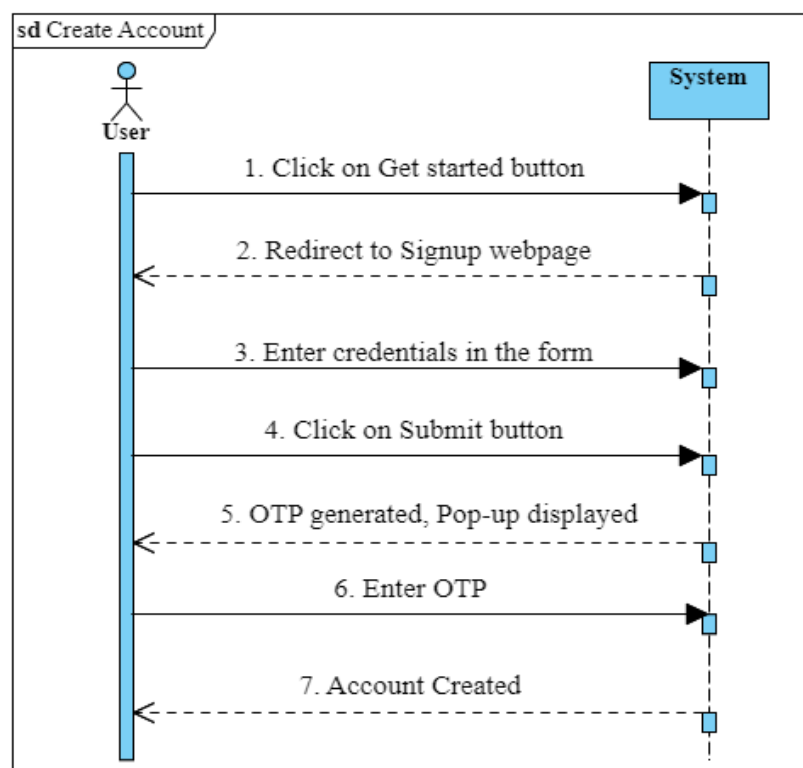


Figure 2.2: SSD on Create Account

Login into Account:

This System Sequence Diagram for logging into an account provides a visual overview of the interactions between the user and the system. It facilitates a clear understanding of the login process from Table 2.5 highlighting the steps involved in user authentication.

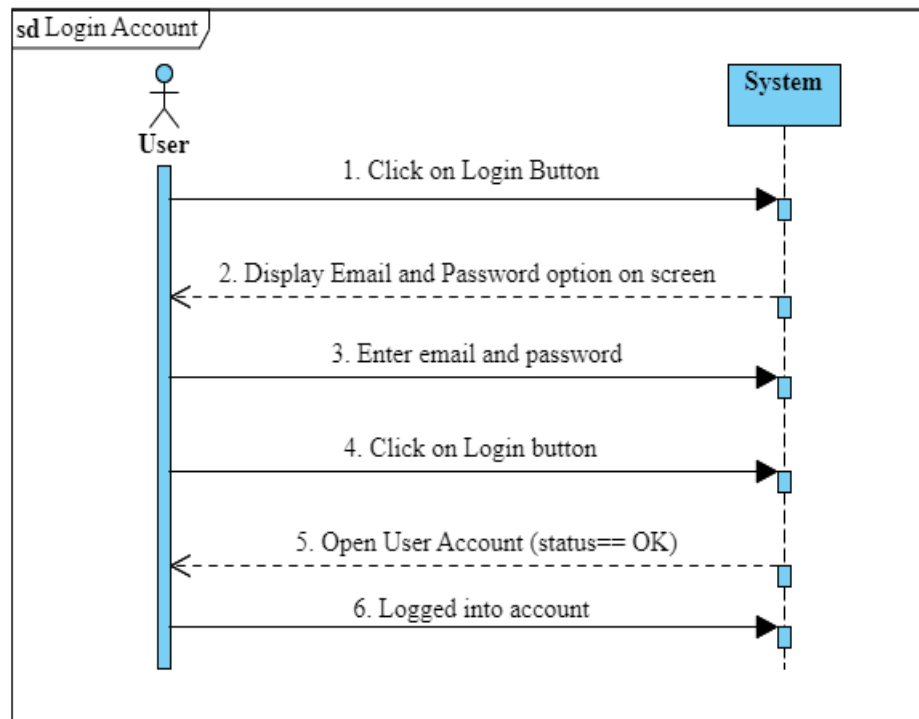


Figure 2.3: SSD on Login Account

Create New Project or Access Previous Projects:

The sequence diagram provides a visual representation of the dynamic interactions from Table 2.6 offers a clear depiction of the interactions between the user and the system, emphasizing the steps involved in creating new project or accessing previous projects.

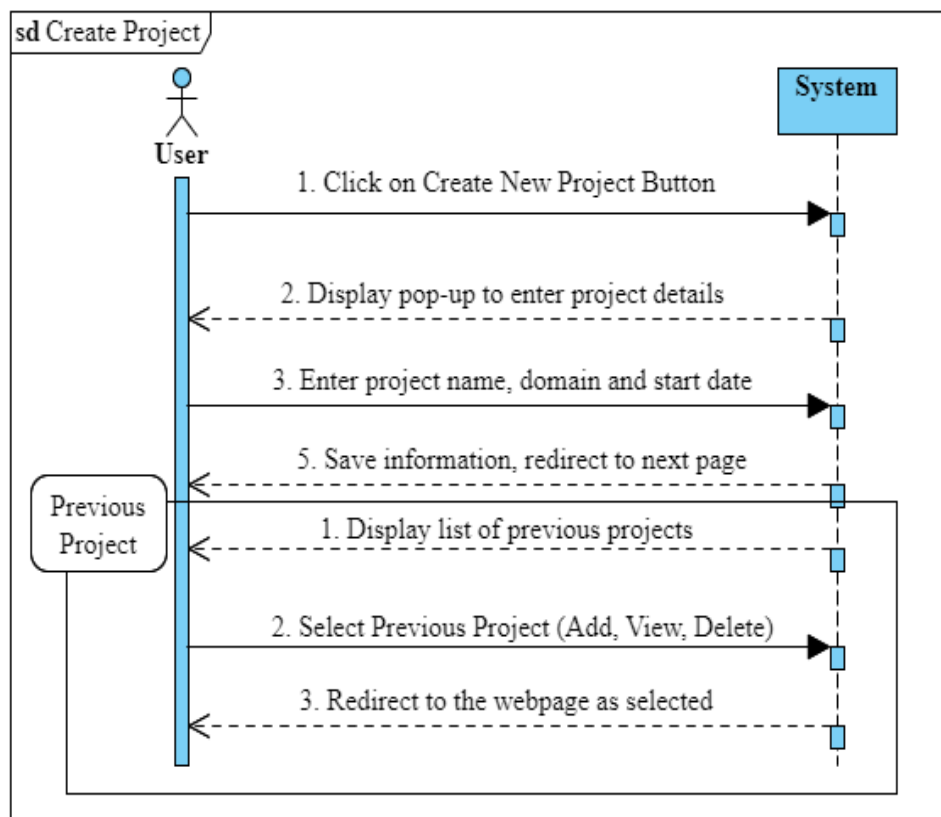


Figure 2.4: SSD on Create New Project or Access Previous Project

Upload Reference Research Paper links (IEEE):

The sequence diagram provides a visual representation of the dynamic interactions from Table 2.7 offers a clear depiction of the interactions between the user and the system, emphasizing the steps involved in uploading webpage links, processing the uploaded links.

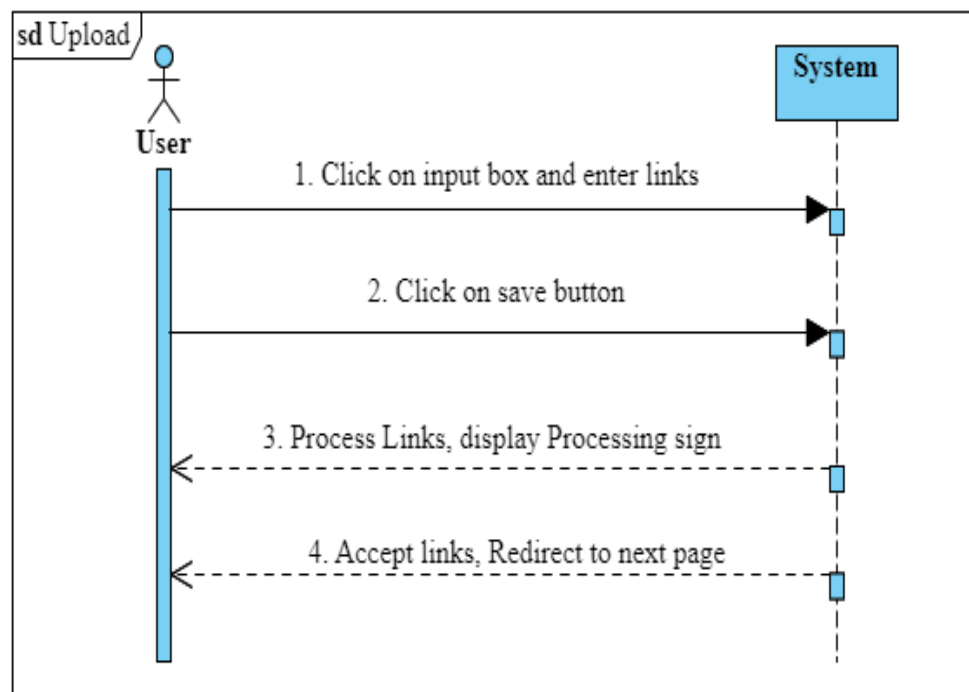


Figure 2.5: SSD on Uploading Reference Research Paper

View Extracted Content:

The SSD in Figure 2.6 illustrates the sequence of interactions when a user views the extracted content of a research paper. The interactions emphasize on the retrieval and display of documents content.

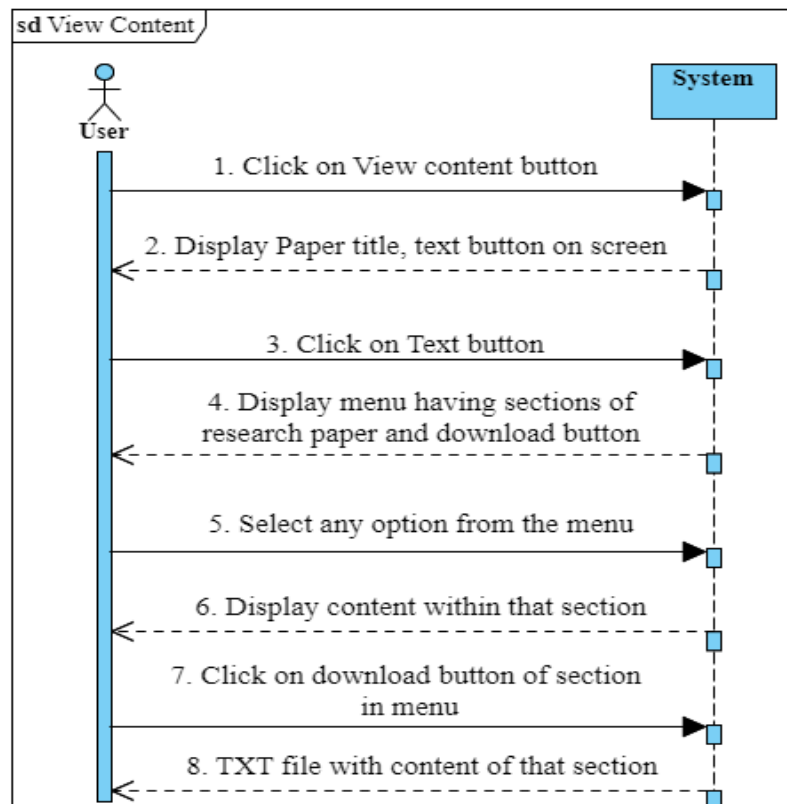


Figure 2.6: SSD on View Extracted Content

Send Prompt to acquire suggestions:

The system sequence diagram in Figure 2.7 provides a visual representation of the dynamic interactions when the user sends prompt to acquire suggestions to the system. These interactions tell the flow between the user send queries and system generates response.

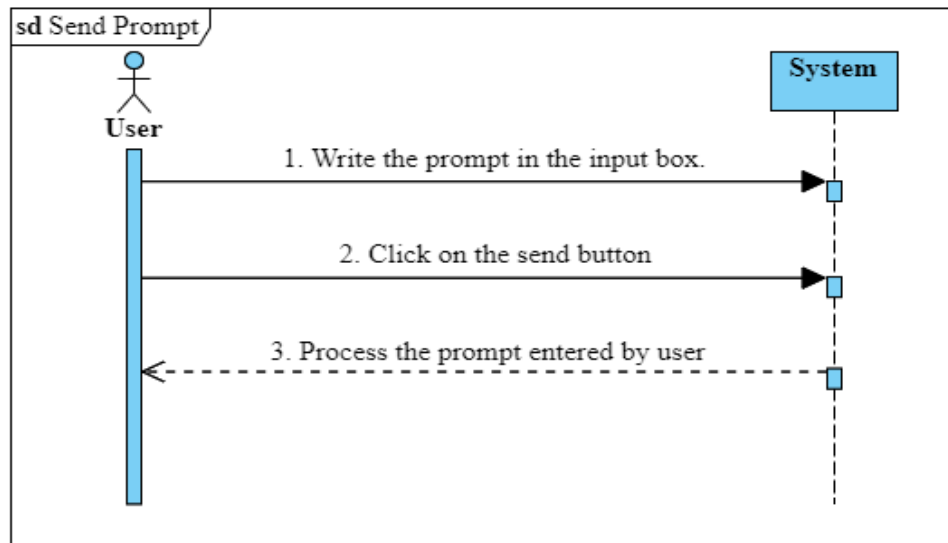


Figure 2.7: SSD on Sending prompt

View Suggestions:

The SSD in Figure 2.8 illustrates the sequence of interactions when a user views suggestion for optimizing a research paper. This System Sequence Diagram for viewing suggestions provides a clear depiction of the interactions between the user and the system.

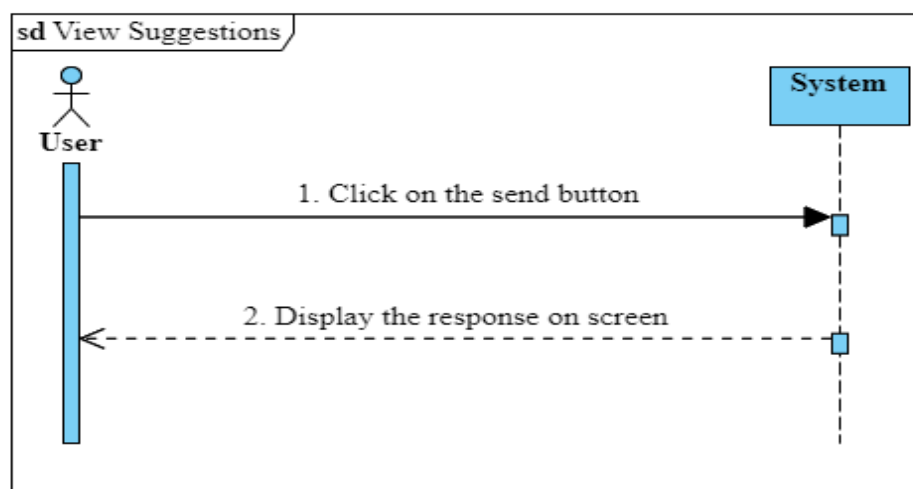


Figure 2.8: SSD on View Suggestions

Edit Text in Text Editor:

The SSD in Figure 2.9 illustrates the sequence of interactions when a user edits text in the text editor for optimizing own research paper content. This System Sequence Diagram for editing text provides a clear depiction of the interactions between the user and the system.

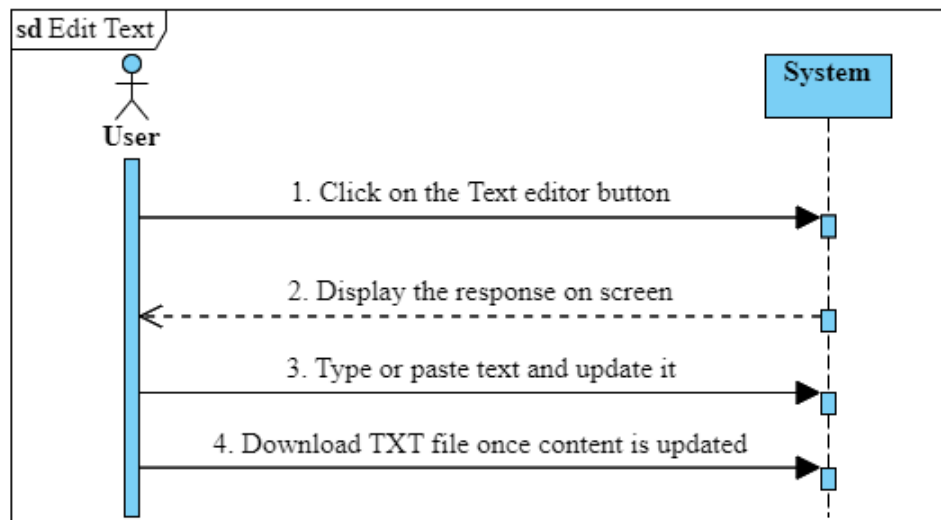


Figure 2.9: SSD on Edit Text in text editor

2.6. Domain Model

Domain model is a conceptual model, used in a way that shows the entities and relationship between them [4]. The diagram in Figure 2.10 is a foundation for understanding how users, research papers, suggestions, and view components are structured and interconnected.

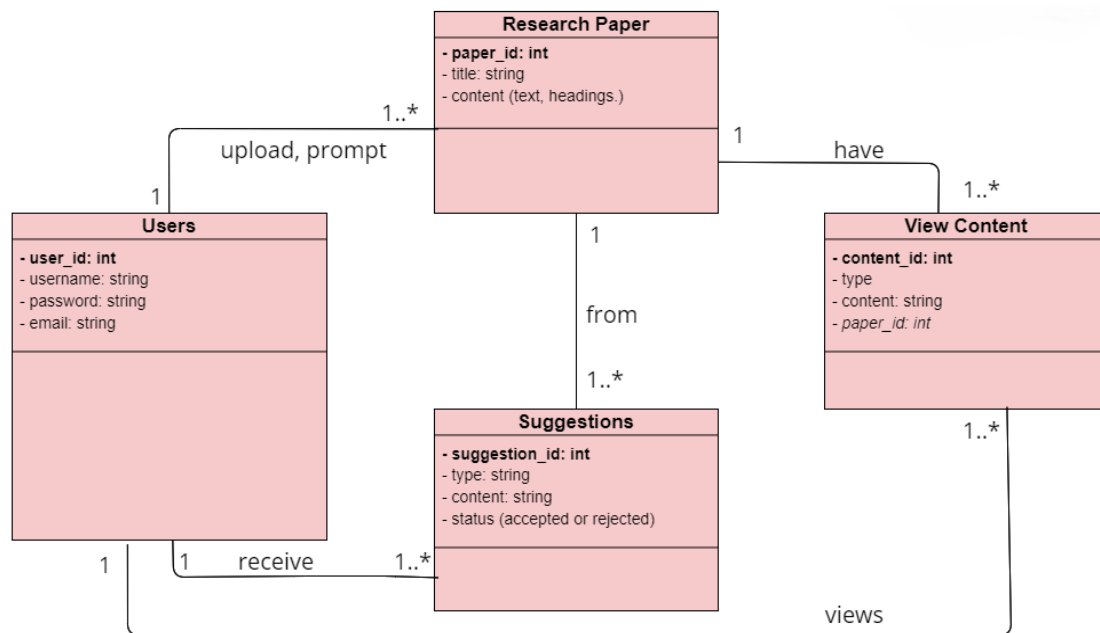


Figure 2.10: Domain Model

Chapter 3

System Design

The process of system design involves the creation and planning of various elements such as modules, architecture, components, interfaces, and data to fulfill the specific requirements of a project. It encompasses the identification and development of systems that align with our project's unique goals.

3.1. Software Architecture

Architecture serves as the blueprint for a system, offering a way to manage its complexity and facilitating communication and coordination among its components [5].

The architecture described below in Figure 3.1 is a hybrid of several architectural styles, and it can be classified as a combination of Client-Server Architecture and Microservices Architecture.

Client-Server Architecture:

The division between the client interface and the server-side elements (web server, application server, services, and system) exemplifies the core principles of client-server architecture.

Microservices Architecture:

Figure 3.1 depicts the existence of dedicated services (such as Upload webpage links, Suggestions Generation, View Content) designed to manage specific tasks. Every service embodies a microservice, which is an architectural approach characterized by small, autonomous, and loosely interconnected components.

The presented architecture incorporates a blend of styles to attain modularity, scalability, and maintainability by utilizing their respective advantages.

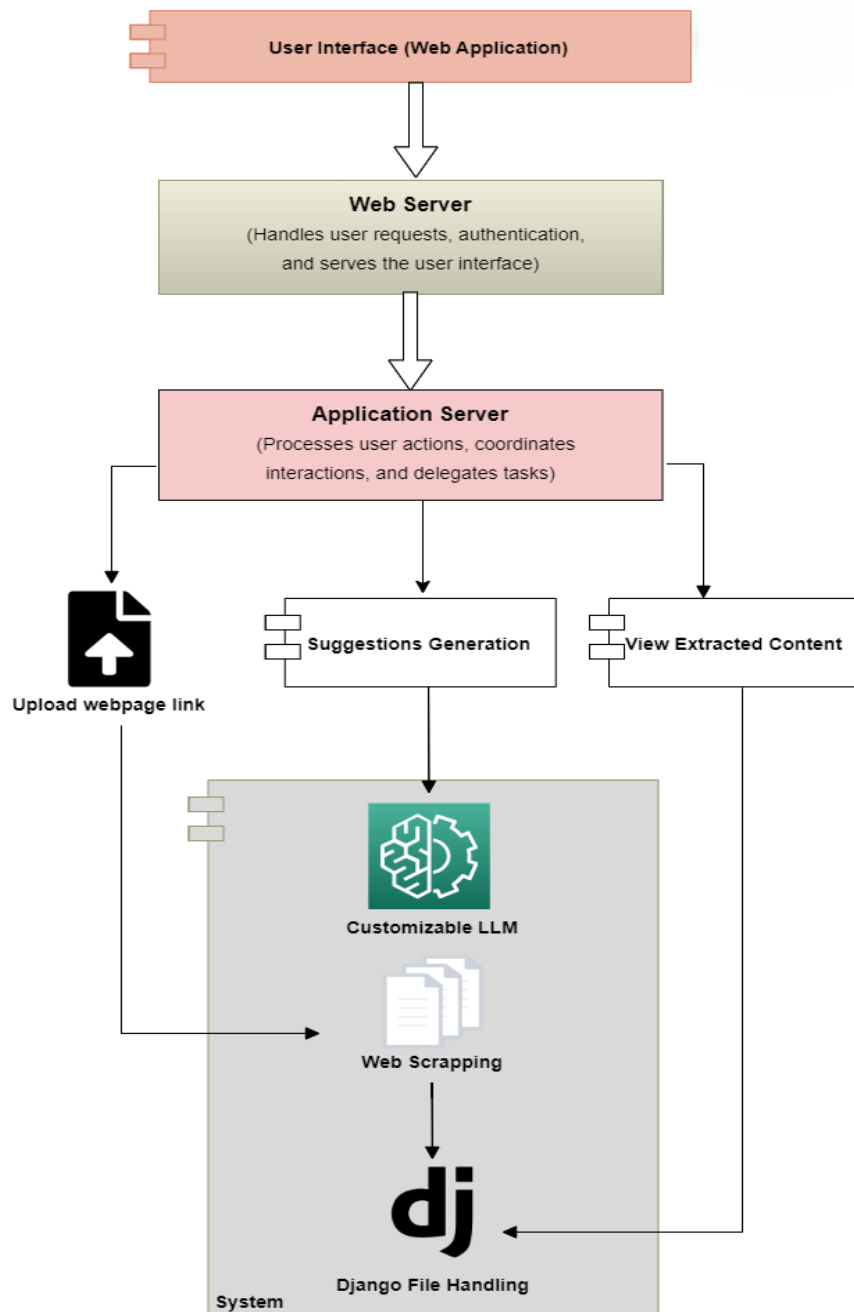


Figure 3.1: System Architecture Diagram

3.2. Class Diagram

The class diagram represents the fixed state of an application, showing the various elements and their connections within the system [4]. It depicts objects grouped into classes, with possibilities for inheritance from other classes. This visual representation serves to showcase, clarify, and document numerous aspects of a system while also facilitating the creation of functional software code.

Following are the classes added in Figure 3.2 which is the project's class diagram:

User:

This class represents a user interacting with our system and includes attributes for user identification as well as methods for accessing user functionalities.

Research Paper:

This class embodies the research papers submitted by a user. It encompasses properties for identifying papers, such as title and sections, alongside content. Its functions enable both the uploading and modification of research papers.

Suggestions:

This class represents suggestions generated by the system to optimize research papers, including attributes for suggestion identification, type, and content. Methods enable the generation of suggestions as well as their respective prompts.

View Content:

This class represents extracted content of a research paper that users can view section wise. It comprises properties for identifying the paper, sections and its content. The functions enable the user to access text from different research papers.

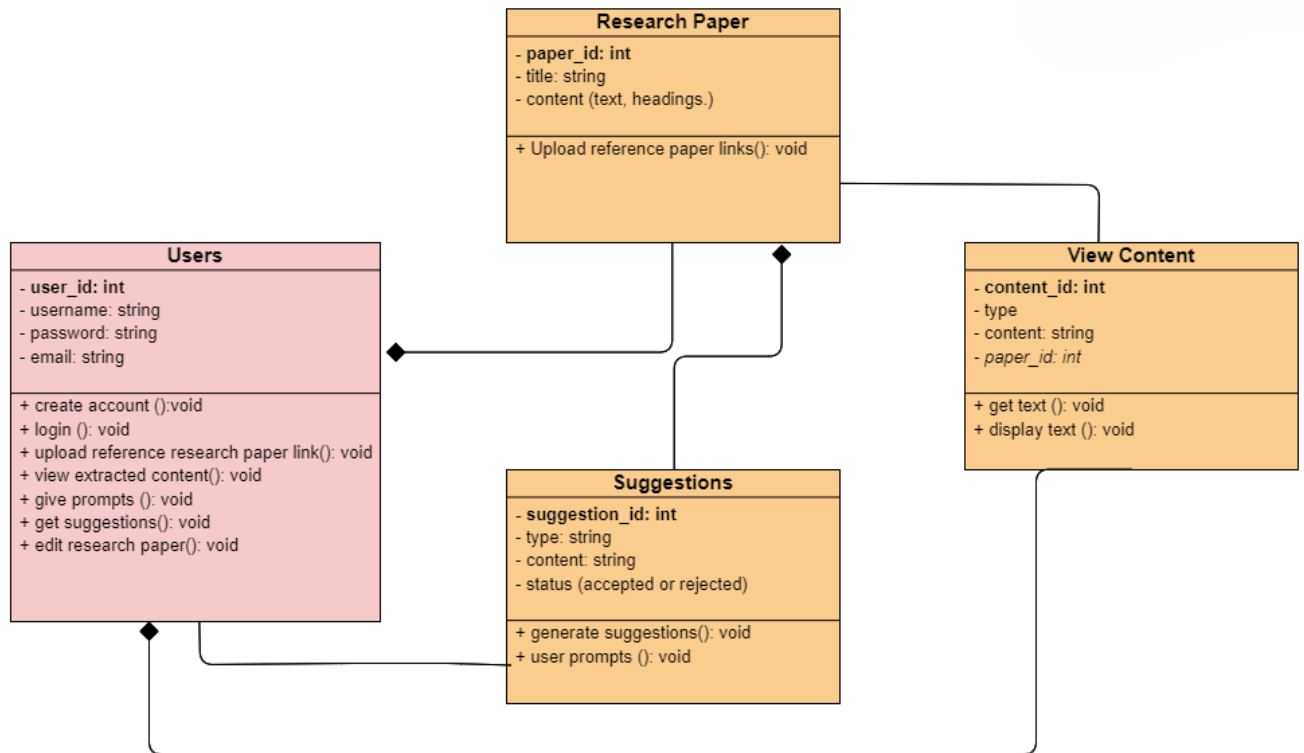


Figure 3.2: Class Diagram

3.3. Sequence Diagram

A sequence diagram simply depicts the interaction between objects in a sequential order, showing the specific order in which, these interactions take place. It describes how and in what order the objects in a system function.

Login Account

The sequence diagram in Figure 3.3 displays the interaction when user creates new account and login into the account in a sequential order.

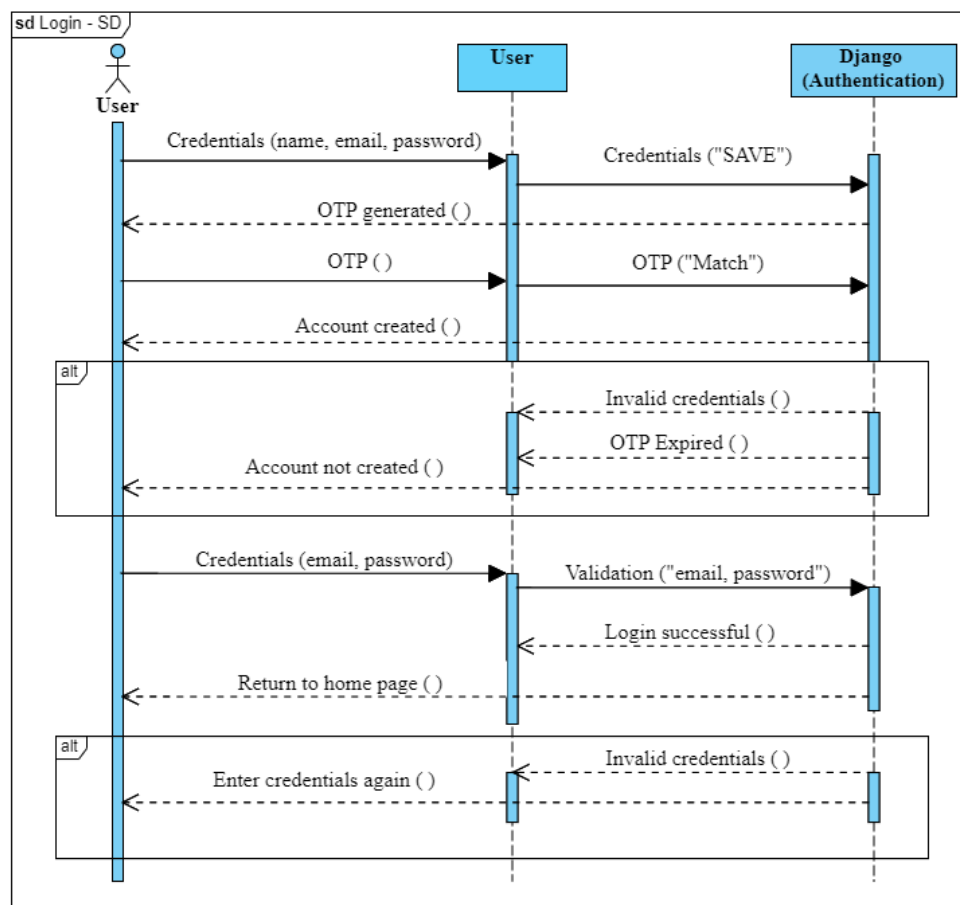


Figure 3.3: Sequence Diagram - Login

Upload Research Paper links (IEEE):

The sequence diagram in Figure 3.4 depicts the interaction in a sequential order when user uploads research paper links.

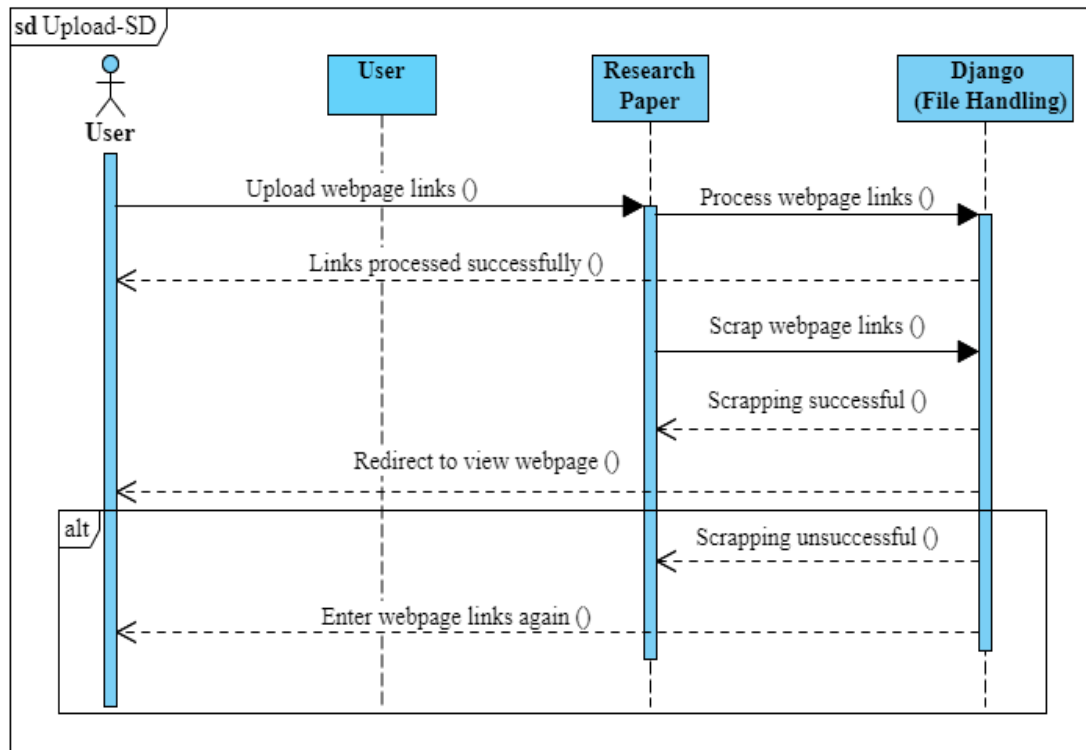


Figure 3.4: Sequence Diagram – Uploading Research Paper links

View Extracted Content:

The sequence diagram in Figure 3.5 displays the interaction when user view extracted content from the uploaded reference research papers.

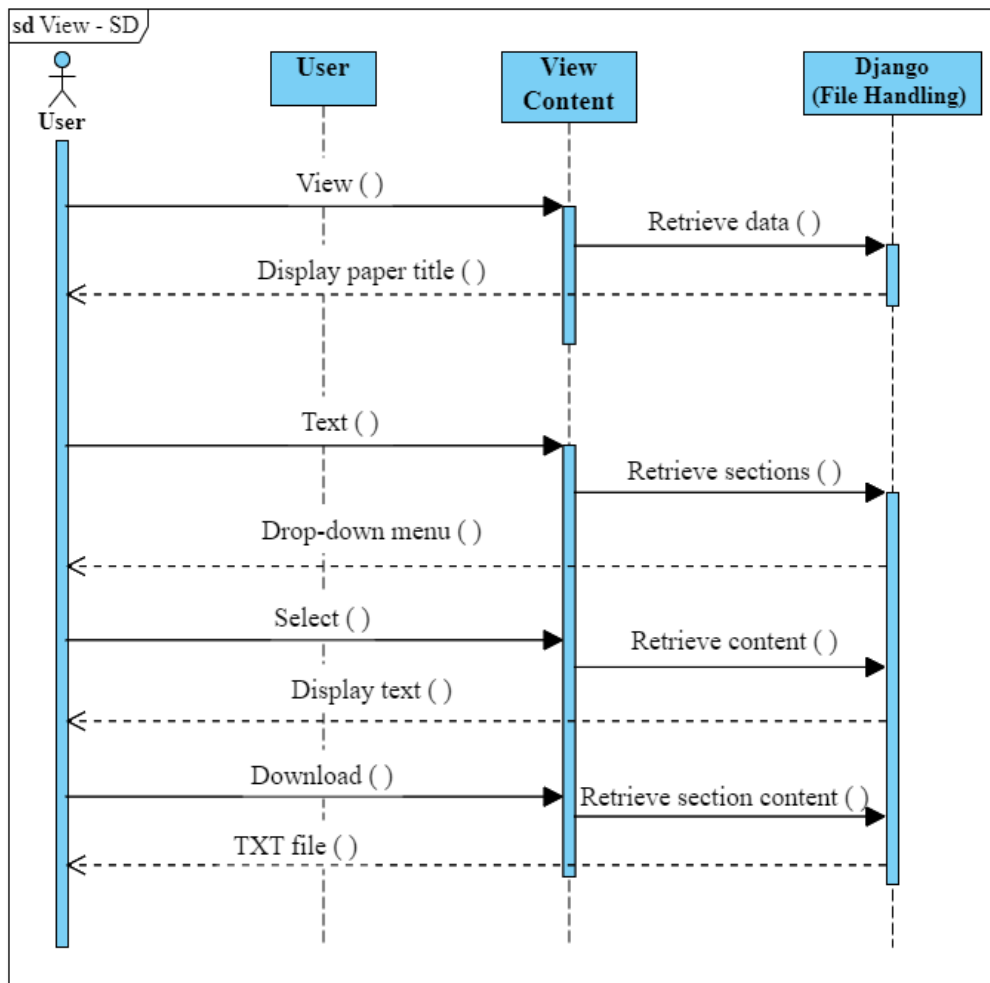


Figure 3.5: Sequence Diagram – Viewing Extracted Content

Suggestions:

The sequence diagram in Figure 3.6 depicts the interaction in a sequential order when user gives prompt and acquire response against the entered prompt.

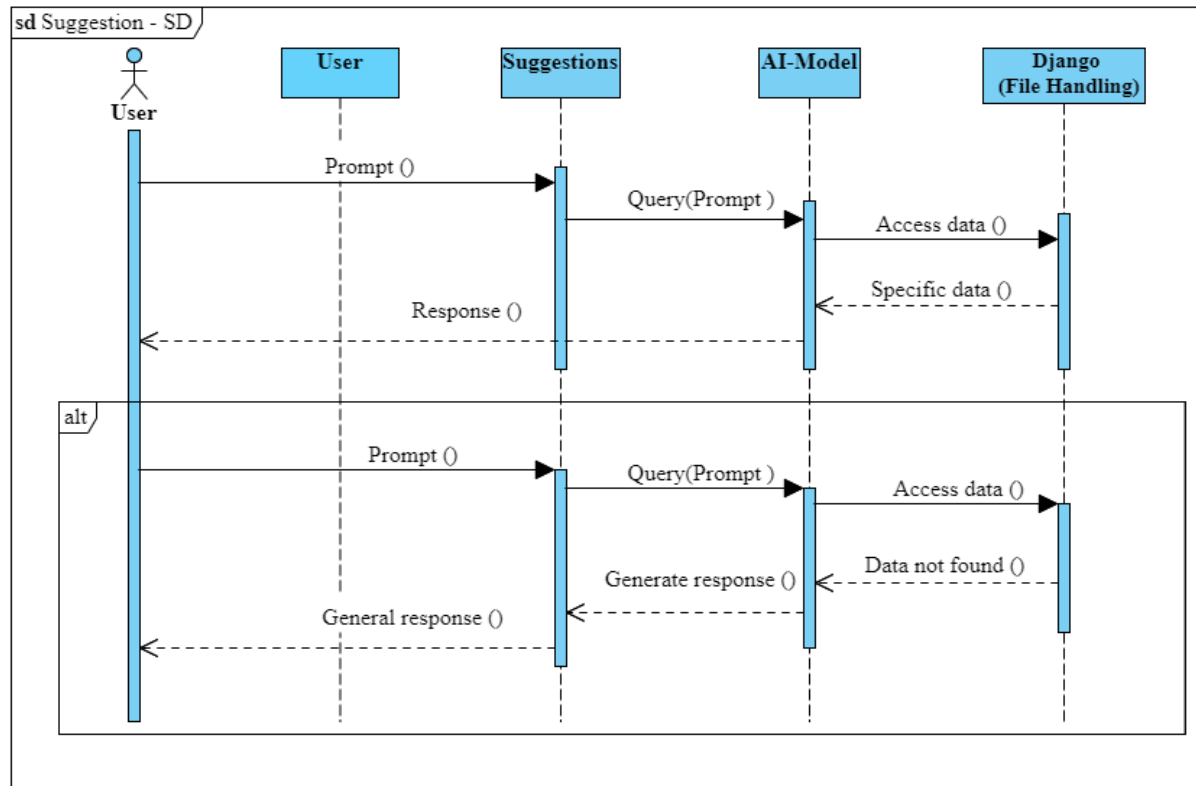


Figure 3.6: Sequence Diagram – Suggestions

3.4. Entity Relationship Diagram

An entity relationship diagram, also referred to as an entity relationship model, is a visual representation of the connections between individuals, objects, places, ideas, or occurrences within an IT system. The ER model represents a data structure that defines how data or information can be organized in a database.

Following are the entities displayed in Figure 3.7 having their primary and foreign keys:

User:

Primary Key: **User_ID**

Foreign Key: *Paper_ID, Suggestion_ID, Content_ID.*

Research Paper:

Primary Key: **Paper_ID**

Foreign Key: *User_ID, Content_ID.*

Suggestions:

Primary Key: **Suggestion_ID**

Foreign Key: *Paper_ID*

View Content:

Primary Key: **Content_ID**

Foreign Key: *Paper_ID*

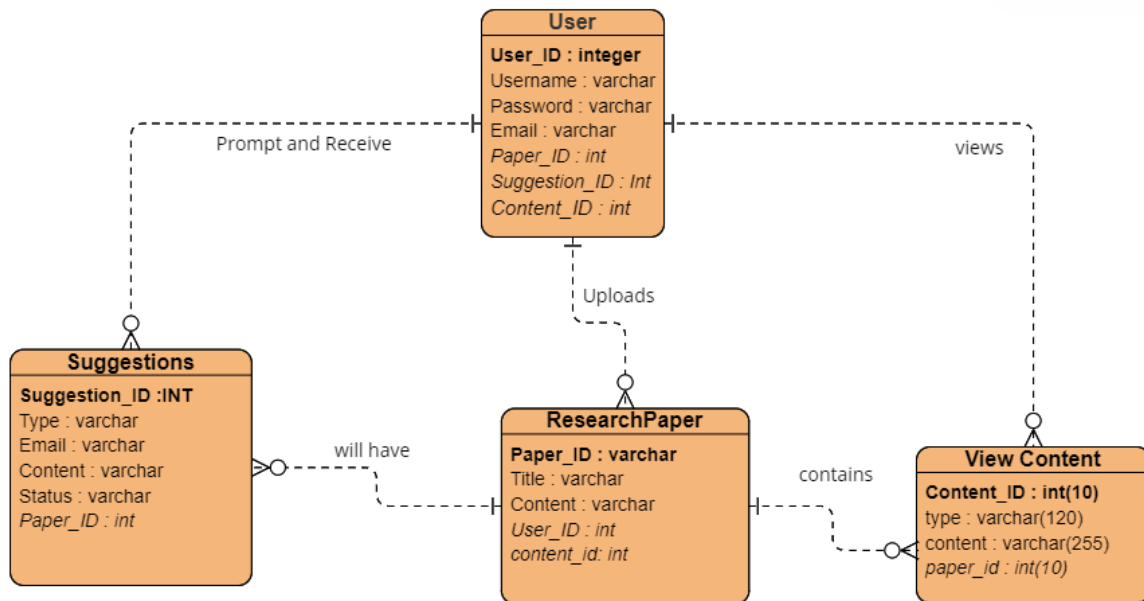


Figure 3.7: Entity Relationship Diagram

3.5. Database Schema

The Schema Diagram in Figure 3.8 visually represents the database structure of the project. Primary keys ensure data integrity and uniqueness by uniquely identifying each record in a table, while foreign keys connect related data across different tables, establishing relationships between them.

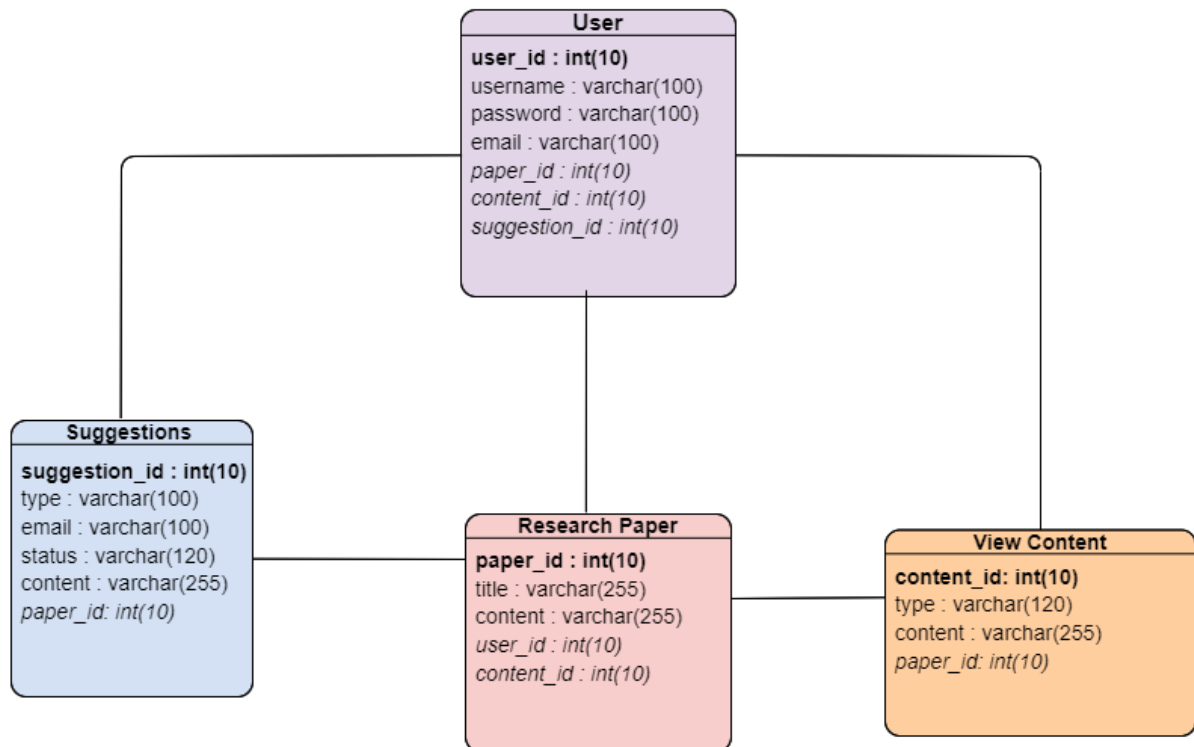


Figure 3.8: Database Schema

In our project, we are not utilizing a traditional database for data storage. Instead, we are leveraging the file handling techniques provided by the Django framework to manage and store our data. This approach allows us to efficiently read, write, and manipulate files directly within the Django server, ensuring seamless integration and flexibility for our web application's data management needs.

3.6. User Interface Design

User Interface Design involves predicting and addressing users' potential tasks, and ensuring that the interface includes accessible, comprehensible elements to support these activities. This encompasses integrating principles from interaction design, visual design, information architecture, and user experience design.

Our website's Graphical User Interface is crafted using HTML, complemented by CSS and JavaScript.

Home Page:

In Figure 3.9, our website's main page is displayed which consists of overview and services of the AI-powered research paper writing tool while potentially highlighting its capabilities.

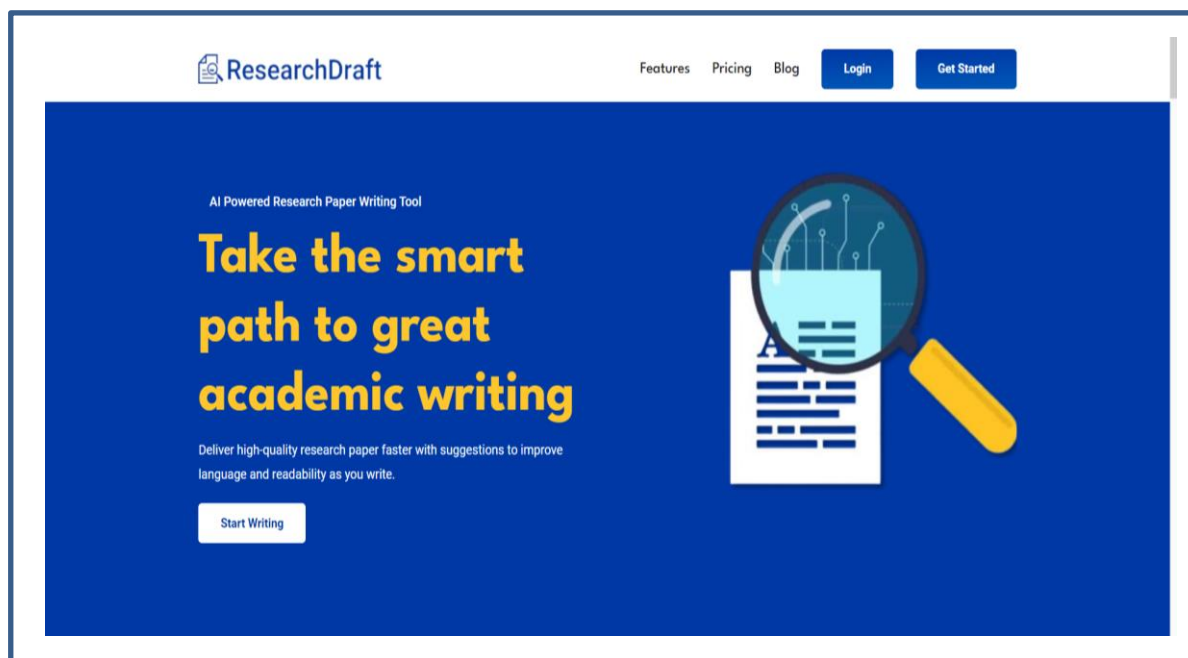


Figure 3.9: Website - Home Page

Features:

In Figure 3.10, Features page is displayed which consists of features and advantages of our website while potentially highlighting its capabilities.

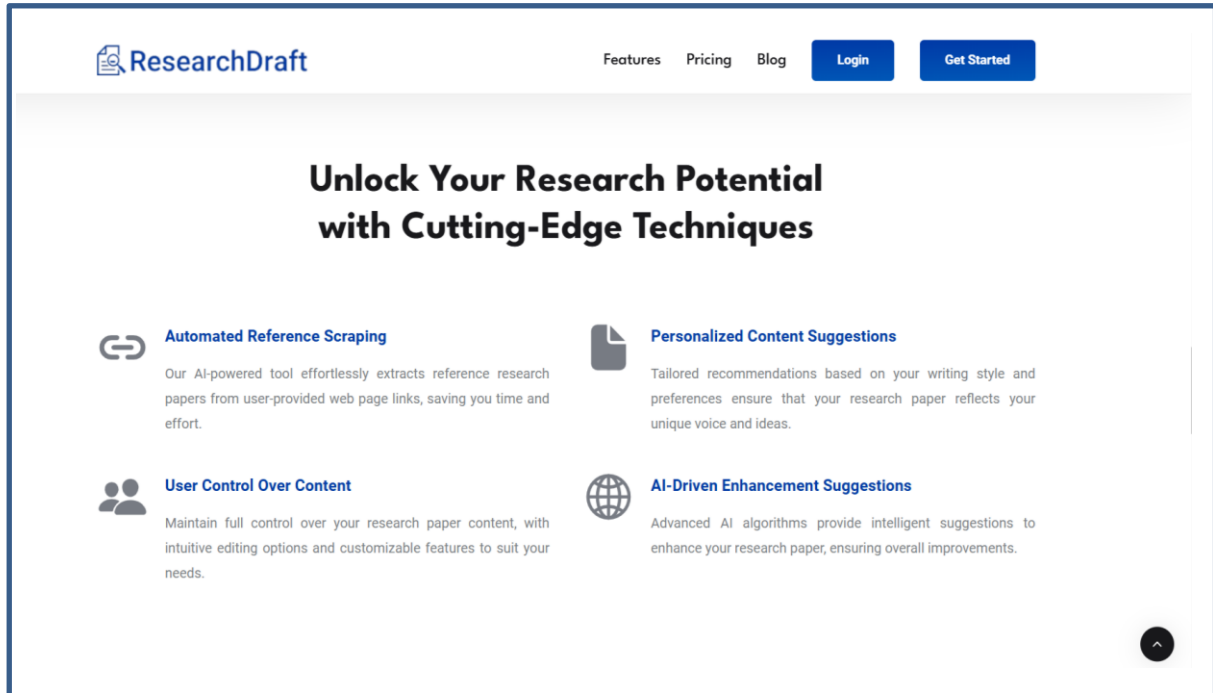


Figure 3.10: Website – Features

Pricing:

In Figure 3.11, Pricing page is displayed which consists of pricing plans available on our website to access the features.

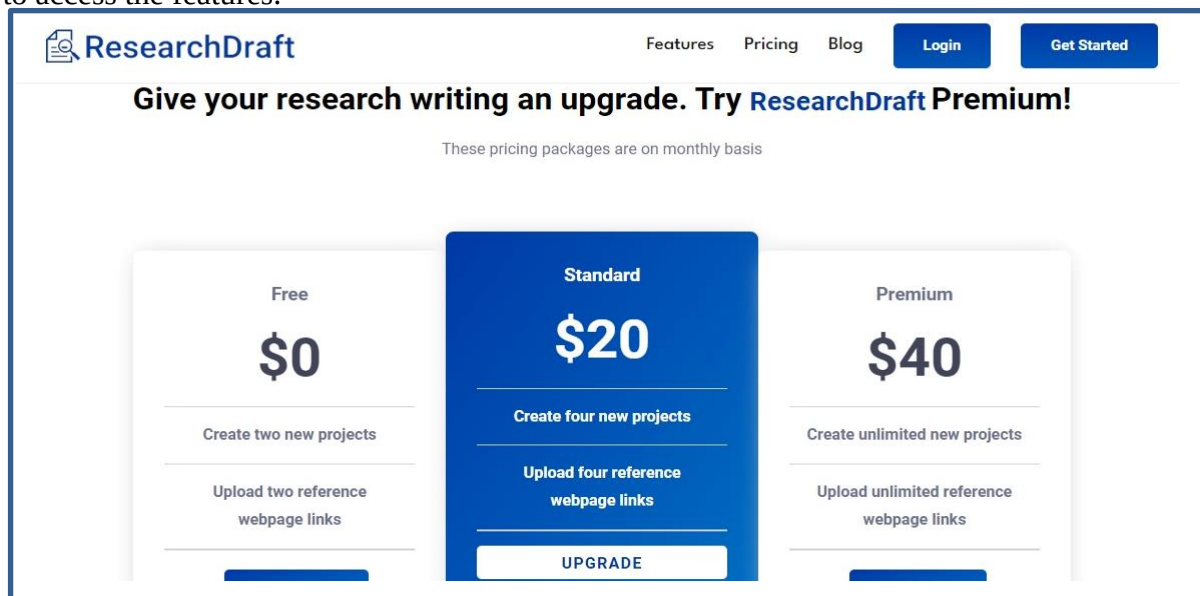


Figure 3.11: Website - Pricing

Blogs:

In Figure 3.12, our website's blog page is displayed which consists of blogs about artificial intelligence and research paper writing.

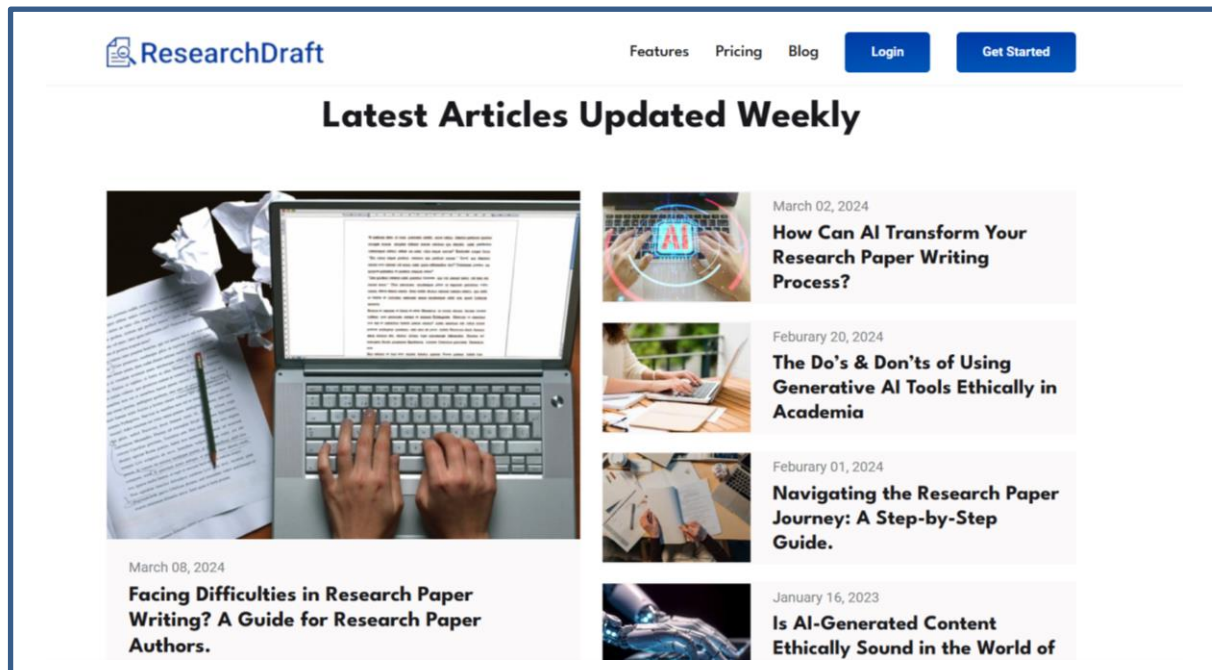


Figure 3.12: Website – Blogs

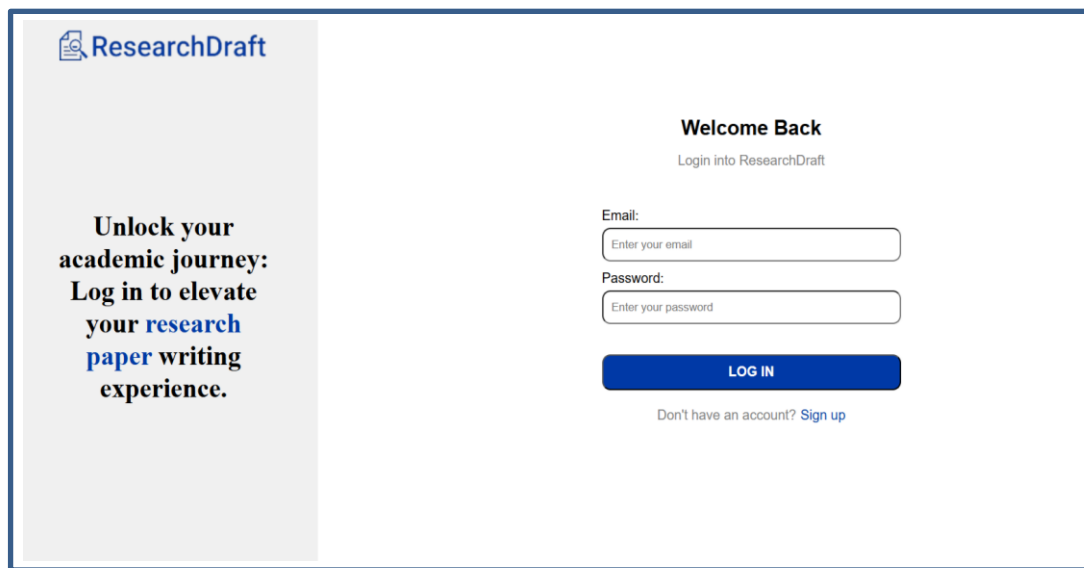
Create Account:

In Figure 3.13, the create account webpage is displayed, we have added little transition to make this page interactive for users as well.

Figure 3.13: Website - Sign up

Log-in:

In Figure 3.14 login page is displayed of the website from which users after entering credentials will be able to access their previous work and other features.

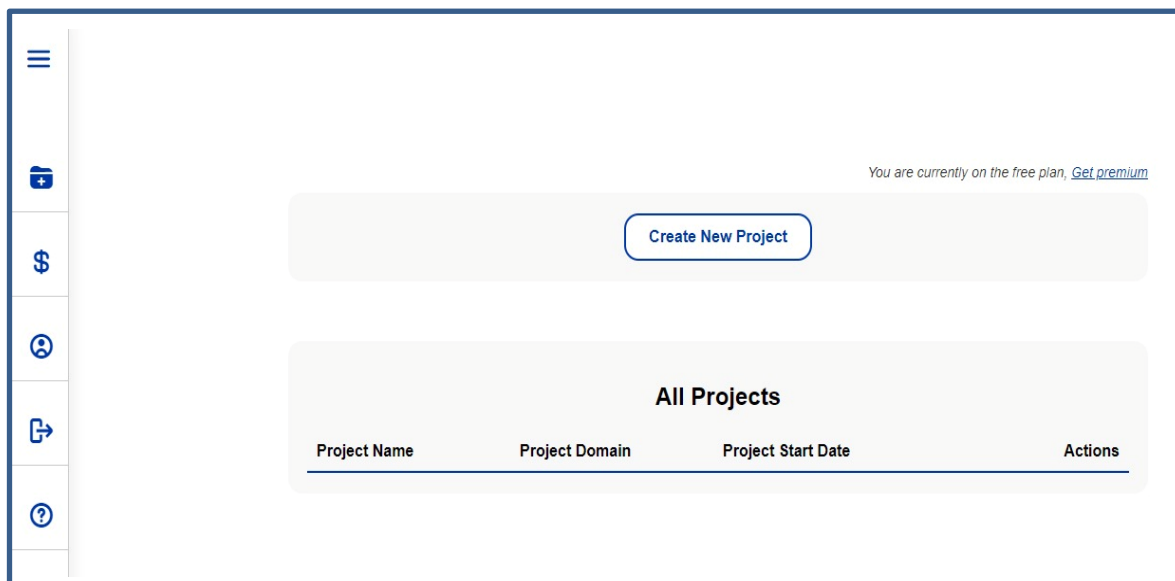


The login page for ResearchDraft features a light gray sidebar on the left with the ResearchDraft logo and a motivational message: "Unlock your academic journey: Log in to elevate your research paper writing experience." The main content area is white and contains a "Welcome Back" heading, a sub-header "Login into ResearchDraft", and two input fields for "Email" and "Password". A blue "LOG IN" button is positioned below the password field, and a link for "Sign up" is at the bottom right.

Figure 3.14: Website – Login

Main dashboard:

In figure 3.15, the enter reference links webpage is displayed in which user will enter reference research paper webpages links (IEEE).



The main dashboard has a vertical sidebar on the left with icons for home, wallet, currency, profile, and help. The main content area is white and includes a "Create New Project" button. Below this is a section titled "All Projects" which contains a table with the following headers: "Project Name", "Project Domain", "Project Start Date", and "Actions". A notification at the top right states "You are currently on the free plan, [Get premium](#)".

Figure 3.15: Website – Main dashboard

Upload Reference Research Paper Links:

In figure 3.16, the enter reference links webpage is displayed in which user will enter reference research paper webpages links (IEEE).

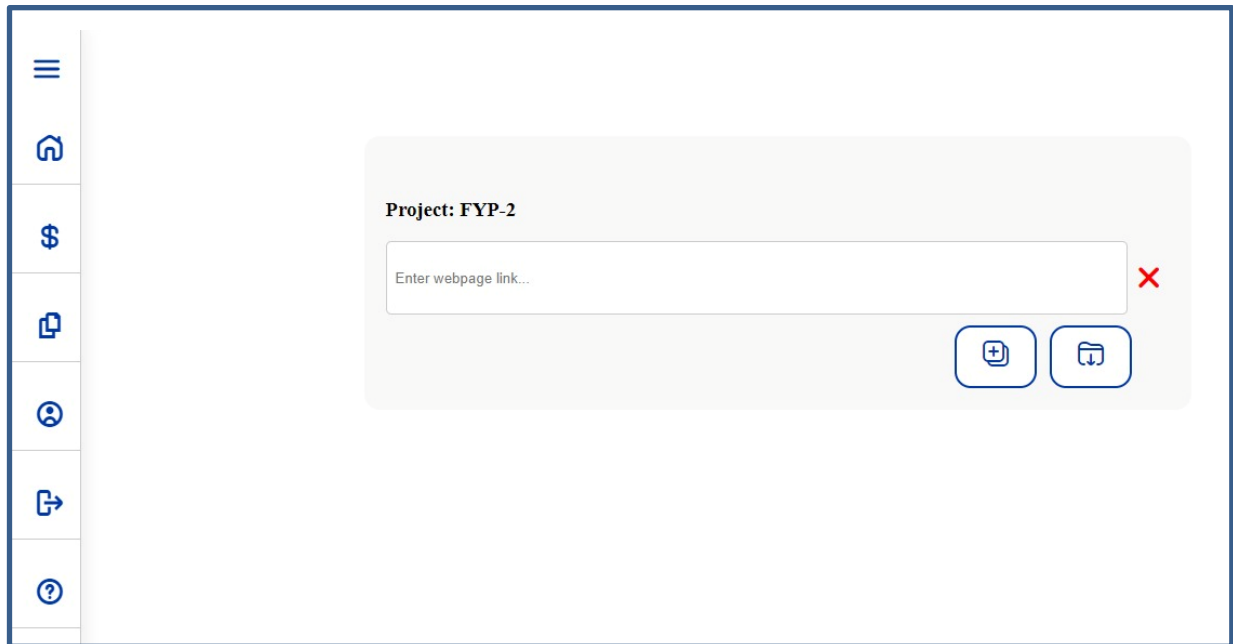


Figure 3.16: Website - Upload Paper links

View Content:

In figure 3.17, the view extracted content webpage is displayed in which user will be able to view content section wise.

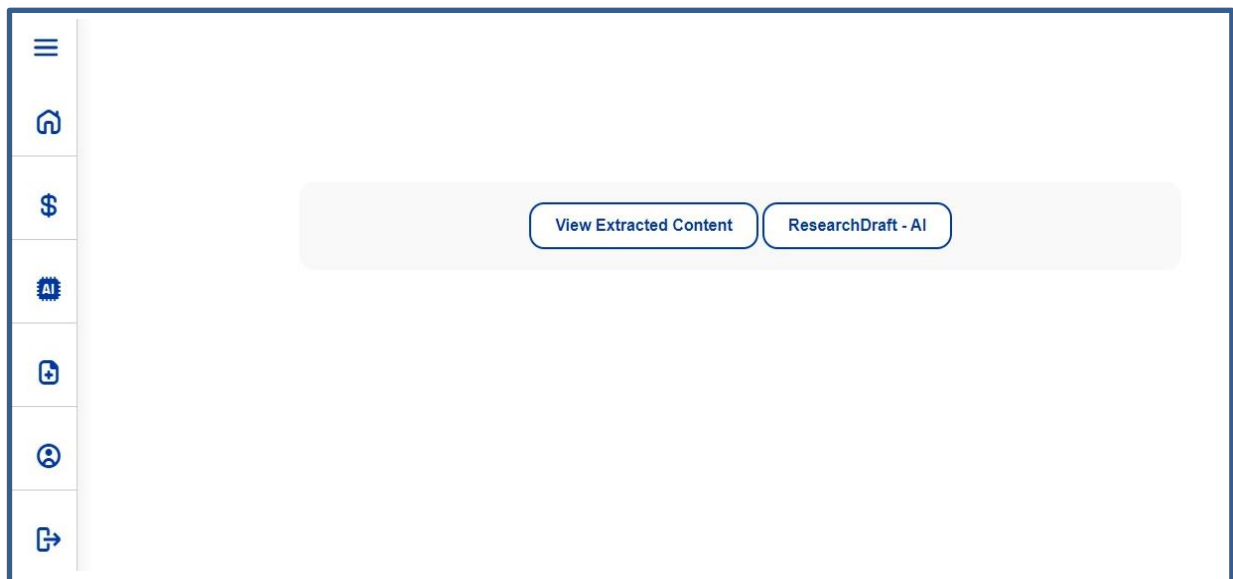


Figure 3.17: Website – View Content

Chapter 4

Software Development

In this chapter, we focus on the practical application of the theoretical framework and design discussed in earlier chapters. This part offers a detailed examination of the development process, offering a deeper understanding of the approaches, tools, and technologies used to implement the suggested solution.

4.1. Coding Standards

For the backend development of our project, we are using the Python programming language. Python has its own style guidelines for the code, one of which is PEP 8 – the official Python style guide. Overall, our code is well-organized and adheres to these guidelines.

Indentation:

The code adheres to the standard Python indentation of four spaces, promoting readability and compliance with PEP 8.

Declaration:

The imports are well-structured, following the practice of grouping standard library imports first, then third-party libraries. One declaration is made per line to improve clarity. Constants in the code adhere to the snake_case convention, a recommended approach in Python naming conventions.

Statement Standards:

Descriptive variable names contribute to improved code legibility. Functions are appropriately defined and aligned with the PEP 8 style guide. Effective use of comments clarifies the purpose of code blocks.

Naming Conventions:

Proper naming convention rules are followed during project implementation, which makes programs more understandable by making them easier to read. It is important to remember to use Natural Language (English) words when naming classes, variables, and methods. Preprocessing, for example, avoided the use of single letter variables.

4.2. Development Environment

The software development for this project primarily utilizes Python and Django, a robust web framework that simplifies the development of web applications. This environment supports multiple programming paradigms, making it ideal for handling diverse tasks such as natural language processing (NLP) and large language model (LLM) integrations.

Visual Studio Code:

This environment is developed by Microsoft, and is a widely used code editor known for its versatility across multiple programming languages and availability of extensions tailored for Python development. We are using this environment as well for coding due to its robust features like syntax highlighting, code completion, and integrated terminal functionality.

The system incorporates a structured workflow for managing user data and projects, enabling users to create, update, and delete projects seamlessly. Secure authentication and session management are facilitated through Django's built-in user authentication system. Data extraction and equation processing are automated to enhance the user experience. This environment ensures secure, efficient and scalable web application development, tailored to the needs of academicians, researchers and students.

4.3. Software Description

Login Process:

This Django view function handles user login by processing form submissions. For `POST` requests, it validates the login form and uses the provided credentials to authenticate the user. If authentication is successful, the user is logged in and redirected to the `create\_project` page. If authentication fails, an error message is displayed. For `GET` requests, an empty login form is displayed. The function ensures a seamless user experience by providing immediate feedback and appropriate redirects based on the authentication outcome.

Model.py:

```
class User(models.Model):
    id = models.AutoField(primary_key=True)
    first_name = models.CharField(max_length=100)
    last_name = models.CharField(max_length=100)
    password = models.CharField(max_length=100)
    email = models.EmailField()
    phone_number = models.IntegerField()
```

View.py:

```
def login_page(request):
    if request.method == 'POST':
        form = LoginForm(request.POST)
        if form.is_valid():
            username = form.cleaned_data['username']
            password = form.cleaned_data['password']
            user = authenticate(request, username=username,
password=password)
            if user is not None:
                login(request, user)
                return redirect('create_project') # Redirect to
dashboard or any other page
            else:
                form.add_error(None, "Invalid username or password")
        else:
            form = LoginForm()
            return render(request, "myApp/login.html")
```

Web Scrapping:

This part of code scrapes content from IEEE Xplore documents. It defines a function `extract_content_and_save` to extract paragraphs and tables from a web page section, saving them into text files. The main function `scrape_ieeeexplore` uses a Chrome web driver to navigate to a specified IEEE Xplore document URL, extracts the document title, creates a corresponding folder, and iteratively extracts and saves content from sections of the document until no more sections are found. The process ensures proper resource management and error handling.

Libraries used:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

IEEE Scraper & Extracted Content Save:

```
def extract_content_and_save(sec_element, folder_path, section_name):
    section_content = f"{section_name}\n\n"
    paragraphs = sec_element.find_elements(By.TAG_NAME, 'p')
    for paragraph in paragraphs:
        section_content += preprocess_text(paragraph.text) + "\n\n"

    tables = sec_element.find_elements(By.TAG_NAME, 'table')
    for j, table in enumerate(tables, start=1):
        table_content = table.text
        table_file_name = f"{section_name} Table {j}.txt"
        with open(os.path.join(folder_path, table_file_name), "w",
encoding="utf-8") as file:
            file.write(table_content)
        print(f"Table {j} under section '{section_name}' saved to
file")

    file_name = f"{section_name.replace('_', ' ').txt" # Replace
underscores with spaces in the filename
    file_path = os.path.join(folder_path, file_name)
    with open(file_path, "w", encoding="utf-8") as file:
        file.write(section_content)

    print(f"Section '{section_name}' content saved to file:
{file_path}")

def scrape_ieeeexplore(url, project_folder):
    driver = webdriver.Chrome()

    try:
        driver.get(url)

        title_element = WebDriverWait(driver, 20).until(
            EC.presence_of_element_located((By.CSS_SELECTOR,
'.document-title')))
        title = title_element.text.strip()
        sanitized_title = sanitize_title(title)

        # Create a folder for the paper inside the project folder
        paper_folder = os.path.join(project_folder, sanitized_title)
        os.makedirs(paper_folder, exist_ok=True)

        print(f"Title of the document for URL: {url}")
        print(sanitized_title)
        print("-" * 50)

        i = 1
        while True:
```

```

sec_id = f"sec{i}"
try:
    sec_element = WebDriverWait(driver, 20).until(
        EC.presence_of_element_located((By.ID, sec_id))
    )

    h2_element = sec_element.find_element(By.TAG_NAME,
'h2')

    section_name = f"{i:02d}
{h2_element.text.strip().replace('_', ' ')}" # Replace underscores
with spaces

    extract_content_and_save(sec_element, paper_folder,
section_name)

    i += 1
except:
    break

except Exception as e:
    print(f"An error occurred for URL {url}: {e}")

finally:
    driver.quit()

```

AI-Model:

This code sets up a document retrieval and conversational interface using OpenAI's GPT-4 model. It begins by loading necessary environment variables, including the OpenAI API key, to access OpenAI's services. The `Document` class is defined to manage documents with their content, identifiers, and metadata. The `UTF8TextLoader` class handles the reading of UTF-8 encoded text files, creating `Document` instances from the file content. The core function, `initialize_chain`, takes a user ID, a selected project, and a base directory, then constructs the project path, verifies its existence, and searches for text files within the directory. It logs the files found and loads their content into `Document` instances.

The system then uses the `OpenAIEmbeddings` class to create text embeddings for these documents and stores them in a vector store managed by the `Chroma` class. The `VectorstoreIndexCreator` builds an index from these embeddings, enabling efficient document retrieval based on text similarity. The `ConversationalRetrievalChain` class sets up a chain that uses GPT-4 and the document retriever to handle user queries, providing relevant document content in response. The `initialize_chain` function returns this configured retrieval chain, which integrates file loading, text embedding, indexing, and conversational AI, creating a comprehensive framework for document retrieval and interaction.

Libraries used:

```
from dotenv import load_dotenv
from openai import ChatOpenAI, OpenAIEmbeddings
from chroma import Chroma
from vectorstore_index_creator import VectorstoreIndexCreator
from conversational_retrieval_chain import
ConversationalRetrievalChain
```

Model:

```
class Document:
    def __init__(self, page_content, doc_id, metadata=None):
        self.page_content = page_content
        self.doc_id = doc_id
        self.metadata = metadata if metadata is not None else {}

    def __repr__(self):
        return f"Document(doc_id={self.doc_id},
metadata={self.metadata})"

class UTF8TextLoader(TextLoader):
    def __init__(self, file_path):
        self.file_path = file_path

    def load(self):
        try:
            with open(self.file_path, 'r', encoding='utf-8') as file:
                page_content = file.read()
                doc_id = os.path.basename(self.file_path)
                logging.info(f"Loaded file: {self.file_path}")
                return [Document(page_content=page_content,
doc_id=doc_id)]
        except Exception as e:
            logging.error(f"Error loading {self.file_path}: {e}")
            return []

def initialize_chain(user_id, selected_project,
base_dir="C:\\Users\\HP\\Desktop\\django\\myProject\\user_data"):
    project_path = os.path.join(base_dir, str(user_id),
selected_project)
    logging.info(f"Project directory: {project_path}")

    if not os.path.exists(project_path):
        error_message = f"Project directory for user_id {user_id} and
project {selected_project} does not exist."
        logging.error(error_message)
```

```

        raise ValueError(error_message)

text_files = [os.path.join(root, file)
               for root, _, files in os.walk(project_path)
               for file in files if file.endswith(".txt")]
logging.info(f"Text files found: {text_files}")

if not text_files:
    error_message = f"No text files found for user {user_id} in
project {selected_project}"
    logging.error(error_message)
    raise ValueError(error_message)
documents = []
for file_path in text_files:
    loader = UTF8TextLoader(file_path)
    content = loader.load()
    if content:
        documents.extend(content)

logging.info(f"Total documents loaded: {len(documents)}")
for doc in documents:
    logging.info(f"Document ID: {doc.doc_id}, Content:
{doc.page_content[:100]}...")

embedding = OpenAIEmbeddings()
vectorstore = Chroma(embedding_function=embedding)
index_creator = VectorstoreIndexCreator(embedding=embedding,
vectorstore_kwargs={"vectorstore": vectorstore})
index = index_creator.from_documents(documents)
logging.info("Index created.")

retriever = index.vectorstore.as_retriever(search_kwargs={"k":
1})
chain = ConversationalRetrievalChain.from_llm(
    llm=ChatOpenAI(model="gpt-4o"),
    retriever=retriever,
)

return chain

```

Chapter 5

Software Testing

In the field of software development, rigorous testing is extremely important. This chapter provides a thorough review of the strength, dependability and effectiveness of the developed software system. This stage is essential in the software development cycle as it guarantees that the software meets specific requirements and functions smoothly in various scenarios [6].

5.1. Testing Methodology

Unit testing has been conducted simultaneously during the development of the project. As we moved towards completion of the project and ensured overall system functionality, multiple other types of testing were conducted to ensure a robust and reliable web application.

Unit Testing:

This is a method to test individual units or components of a software application in isolation from the rest of the system. The main aim is to confirm that each unit performs as intended. A "unit" refers to the smallest testable part of an application, such as a function, method, or procedure.

Blackbox Testing:

Blackbox testing is a software testing method that evaluates an application's functionality without examining its internal code or implementation details. The focus is on input and output to ensure the software behaves as expected.

End-to-End (E2E) Testing:

End-to-End (E2E) testing is a testing methodology that assesses the entire workflow of an application from start to finish. The application is tested with real user scenarios to ensure it functions correctly and meets business requirements.

Initially, we performed unit testing to ensure individual components worked correctly. As functionalities developed, we conducted Black box testing to validate system behavior without focusing on internal code. Once the application was completed, we carried out End-to-End testing to verify the entire workflow operated seamlessly, simulating real user scenarios.

5.2. Testing Environment

For the test environment we have used

a. Windows 10 Build No. 20H2 Operating System

Windows 10 was running as an Operating System of our Machine on which we were performing test cases.

b. Visual Studio 2019 Community Edition Version 16.9

We have used Visual Studio to compile the code of the system.

c. Local-host Server

We have used localhost Server as a Server on which our system was deployed and running.

d. Google Chrome Version 89.0.4389.72

We have used Google Chrome as a browser to load web pages of the system.

5.3. Testing Cases

A test case is a specification of the inputs, execution conditions, testing procedure and expected results that define a single test to be executed to achieve a particular software testing objective.

Test Case 1:

In Table 5.1, it is verified that a user can create an account with a username, password, and email address.

Description:

Enter valid username, email, and password and submit the form. Verify that the account is created successfully and a confirmation message is displayed.

Table 5.1: Test Case 1: Create Account

Date: 22/12/2023	
System: Research Draft	
Objective: Create Account	Test-ID: 01
Version: 01	Test Type: Unit Testing
Input: 1. Email: Hamza@gmail.com 2. Password: Hamza123 3. First Name: Muhammad	

4. Last Name: Hamza
5. OTP: 9yt54
Expected Result: Account created successfully and redirected to Login page
Actual Result: Passed

Test Case 2:

In Table 5.2, it is verified that a user can log in with valid credentials.

Description:

Enter valid username and password, then submit the form. The credentials are matched and then the user is redirected to the dashboard.

Table 5.2: Test Case 2: Login Account

Date: 25/12/2023	
System: Research Draft	
Objective: Login into Account	Test-ID: 02
Version: 01	Test Type: Unit Testing
Input: 1. Email ID: Hamza@gmail.com 2. Password: Hamza123	
Expected Result: Login successful, redirect to home webpage.	
Actual Result: Passed	

Test Case 3:

In Table 5.3, it is verified that a user can create a new project.

Description:

Click on the create new project button. Enter project details and submit the form. Verify that the project is created and listed in the user's dashboard.

Table 5.3: Test Case 3 - Create new project

Date: 20/06/2024	
System: Research Draft	
Objective: Create new project	Test-ID: 03
Version: 01	Test Typing: Unit Testing
Input: 1. Project name: Research paper 1 2. Project domain: Computer Science 3. Project start date: 20/06/24	
Expected Result: Project will be created and listed in the All Projects section. User will be redirected to Reference webpage.	
Actual Result: Passed	

Test Case 4:

A user can upload reference research papers via web page links is verified in Table 5.4.

Description:

Upload valid web page links (IEEE) for reference research papers and verify that the links are accepted and saved.

Table 5.4: Test Case 4 – Upload reference webpage links

Date: 23/04/2024	
System: Research Draft	
Objective: Upload reference webpage links.	Test-ID: 04
Version: 01	Test Typing: Unit Testing
Input: Enter reference IEEE webpage links. Example: https://ieeexplore.ieee.org/document/6482532	
Expected Result: Webpage links will be saved and user will be redirected to the View webpage.	
Actual Result: Passed	

Test Case 5:

In Table 5.5, it is verified that a user can view extracted content from the uploaded research papers.

Description:

View extracted content from the uploaded reference research papers and verify that the content is displayed section wise and can be downloaded.

Table 5.5: Test Case 5 – View Extracted Content

Date: 20/03/2024	
System: Research Draft	
Objective: View extracted content from reference papers.	Test-ID: 05
Version: 01	Test Typing: Black Box Testing
Input: Reference research paper webpage links (IEEE). Example: https://ieeexplore.ieee.org/document/6482532	
Expected Result: Section wise content from research papers is displayed and can be downloaded as a TXT file.	
Actual Result: Passed	

Test Case 6:

A user can send queries to acquire AI-driven suggestions is verified in Table 5.6.

Description:

Enter a prompt based on the uploaded reference papers and submit. Verify that the system processes the prompt.

Table 5.6: Test Case 6 – Send prompt

Date: 20/06/2024	
System: Research Draft	
Objective: Send prompt to acquire suggestions.	Test-ID: 06
Version: 01	Test Typing: Unit Testing
Input: Write and send query in the input box. Example: What is Artificial Intelligence? Tell me about branches of computer science?	
Expected Result: System will accept the prompt and process it.	
Actual Result: Passed	

Test Case 7:

In Table 5.7, it is verified that a user can view suggestions for improvement generated by the system.

Description:

After submitting the prompt, check the response generated by the system suggestions. Verify that the AI-driven suggestions are displayed based on the uploaded reference papers if the prompt was related to the reference papers.

Table 5.7: Test Case 7 – View Suggestions

Date: 21/05/2024	
System: Research Draft	
Objective: View AI-driven suggestions	Test-ID: 07
Version: 01	Test Typing: Black box Testing
Input: Prompt entered by the user. Example: What is Artificial Intelligence? Tell me about branches of computer science?	
Expected Result: System generates response according to the prompt.	
Actual Result: Passed	

Test Case 8:

A user can type or paste research paper content in the text editor and edit it and then download it as a TXT file is verified in Table 5.8.

Description:

Navigate to the text editor, type or paste research paper content. Edit the content according to the AI driven suggestions and verify that after the changes the text can be downloaded as a TXT file.

Table 5.8: Test Case 8 – Edit research paper content

Date: 13/06/2024	
System: Research Draft	
Objective: Edit research paper content in the text editor.	Test-ID: 08
Version: 01	Test Typing: Unit Testing
Input: Write or paste text from research paper to edit it.	
Expected Result: Edited content is downloaded as a TXT file.	
Actual Result: Passed	

Chapter 6

Software Deployment

The software deployment chapter's purpose is to document and outline the process of deploying developed software solution to the intended target environment [8]. This chapter provides a detailed description of the steps involved in making the software solution operational, such as configuring hardware and software resources, installing it, integrating it with existing systems(if applicable), and performing any necessary testing and verification. Its goal is to ensure that the software solution can be implemented and used successfully by end users.

6.1. Installation / Deployment Process Description

Front-end Deployment:

To deploy the frontend of our project on Heroku, we followed these steps:

1. Installed Heroku CLI:

We began by downloading and installing the Heroku Command Line Interface (CLI) from the official Heroku website. This tool allows us to manage and deploy our applications directly from the terminal.

2. Logged into Heroku:

Using the terminal, we logged into Heroku. This step is essential to authenticate and link our local project with our Heroku account.

3. Prepared Our Project:

We ensured that our frontend project was ready for deployment. Since our project uses a JavaScript framework, we built the project to generate the static files needed for deployment. This step transforms our source code into a production-ready format.

4. Created a New Heroku App:

We navigated to our project directory in the terminal and created a new Heroku application. This process involved selecting a unique name for our app and establishing a new Heroku environment to host our frontend.

5. Configured Static Site Hosting:

We set up the necessary configuration to host our static site on Heroku. This involved creating a configuration file that Heroku uses to serve our built project files correctly.

6. Deployed the Project:

Finally, we deployed our frontend project to Heroku. This step included pushing our local project files to Heroku's remote server, where they were built and hosted. Once the deployment was complete, we were able to access our application via the unique URL provided by Heroku.

By following these steps, we successfully deployed our frontend on Heroku, ensuring our web interface is accessible and operational on the cloud platform.

Back-end Deployment:

To deploy the backend of our project on Heroku, we followed these steps:

1. Installed Necessary Dependencies:

We started by ensuring that all necessary dependencies and packages required for our backend application were included in the requirements.txt file. This step is crucial for Heroku to install all the needed libraries when deploying our application.

2. Configured Environment Variables:

We set up environment variables in Heroku to manage sensitive information such as API keys and database URLs. This allowed us to keep our configuration secure and separate from our codebase.

3. Configured the Procfile:

We created a Procfile in our project directory, which instructed Heroku on how to run our application. This file specifies the commands needed to start our backend server and is essential for proper deployment.

4. Deployed the Backend Application:

We used the Heroku CLI to deploy our backend application. This process included pushing our code to Heroku, where it was built and set up in the cloud environment. We monitored the deployment process to ensure that there were no errors.

5. Tested the Backend Functionality:

After successfully deploying our backend, we tested the API endpoints to ensure that they were functioning correctly. This included verifying that the backend could connect to any databases or services it relied on.

By following these steps, we successfully deployed the backend of our project on Heroku, allowing us to leverage a scalable cloud environment for our application's server-side functionalities. This deployment ensured that our web application operates seamlessly, providing essential services to the frontend interface.

Website URL: <http://researchdrafts-2d1abe4095bd.herokuapp.com/>

Chapter 7

Project Evaluation

The report of the project, “ResearchDraft: AI-Powered Research Paper Writing Tool” has been approved based on the following evaluation guideline.

Project Evaluation Guidelines

Artifacts Guidelines	
Analysis and Design artifacts are syntactically correct (use-case model, SSDs, domain model, class diagram, SDs, ERDs, Flow charts, Activity Diagram, DFDs)	
Consistency and traceability have been maintained among different artifacts	
General Guidelines	
Formatting (font style, indentation) is according to the FYP template and consistent throughout the document	
Captions are added to all the figures and tables. Figure captions must be placed below each figure, and table captions must be provided above the table	
Each figure or table is followed by some text describing what it represents	

Ms. Rida Haya

Ms. Samra Naseer

Ms. Laraib khan

Dr. Shafiq Ur Rehman Khan

References

- [1] SUNY Empire State College, n.d., Online Writing Support: Research Paper, accessed 01 October 2023, <https://sunyempire.edu/online-writing-support/resources/research/research-paper/>.
- [2] ScienceDirect, n.d., Requirement Specification, accessed 20 October 2023, <https://www.sciencedirect.com/topics/engineering/requirement-specification>.
- [3] Course Hero, n.d., Functional and Non-Functional Requirements , accessed 25 October 2023, <https://www.coursehero.com/file/102685120/18-Functional-and-Non-Functional-Requirementspptx/>
- [4] Chursin, O., 2021, A Brief Introduction to Domain Modeling, accessed 25 November 2023, <https://olegchursin.medium.com/a-brief-introduction-to-domain-modeling-862a30b38353>.
- [5] Course Hero, n.d., Software Architecture, accessed 05 December 2023, <https://www.coursehero.com/file/9703185/Software-architecture/>.
- [6] GeeksforGeeks, n.d., Types of Software Testing, accessed 20 January 2024, <https://www.geeksforgeeks.org/types-software-testing/>.
- [7] IBM, n.d., Large Language Models (LLM), accessed 25 February 2024, <https://www.ibm.com/topics/large-language-models>.
- [8] GeeksforGeeks, n.d., Software Deployment in Software Development, accessed 20 June 2024, <https://www.geeksforgeeks.org/software-deployment-in-software-development/>.

Plagiarism Report

M HAMZA KHALID KIANI

ORIGINALITY REPORT

20%	16%	4%	11%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	www.coursehero.com Internet Source	10%
2	Submitted to Higher Education Commission Pakistan Student Paper	2%
3	Submitted to Middle East College of Information Technology Student Paper	1%
4	dev.to Internet Source	1%
5	Submitted to CSU, Fullerton Student Paper	<1%
6	Submitted to Camarines Sur Polytechnic Colleges Student Paper	<1%
7	Submitted to CSU, San Jose State University Student Paper	<1%